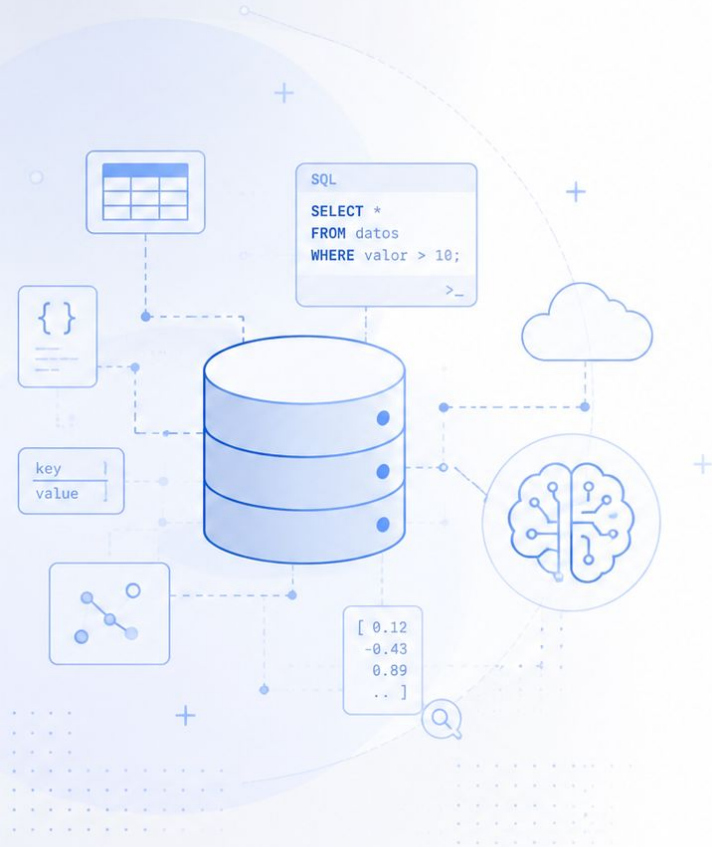




Bases de Datos para Inteligencia Artificial

Especialización en Inteligencia Artificial

Docente:

Lacheski, Martín Anibal

Clase X:

Título

Bases de Datos para IA

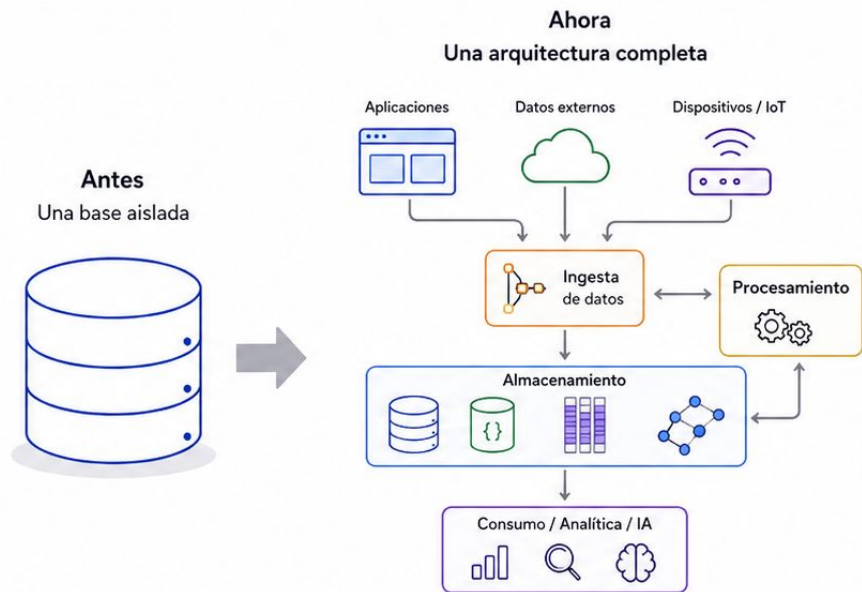
De una base de datos a una arquitectura completa

En las clases anteriores analizamos:

- bases relacionales;
- diseño y normalización;
- consultas SQL;
- bases documentales;
- bases clave-valor;
- bases columnares;
- bases de grafos.

En esta clase vamos a ampliar la mirada.

Ya no vamos a pensar solamente dónde guardar un dato, sino **cómo organizar todo su recorrido**.

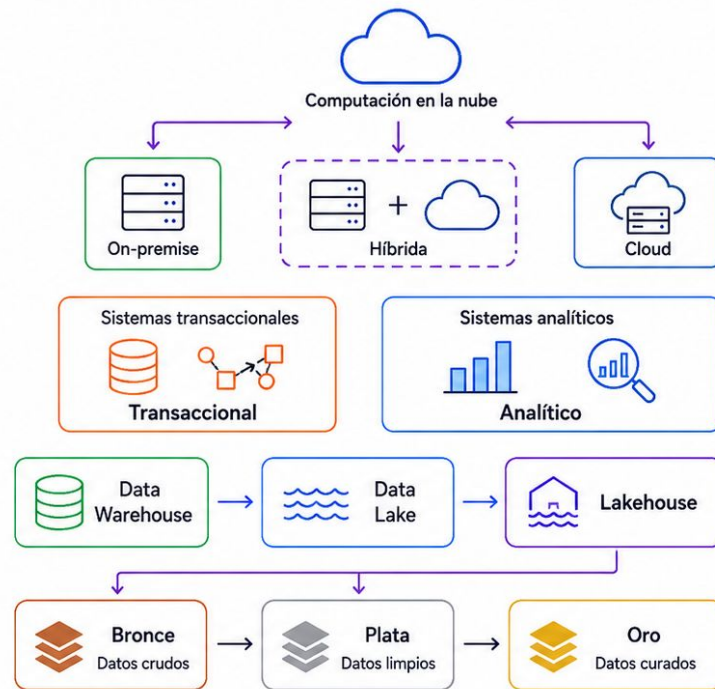


Una base de datos es una pieza.
La arquitectura define cómo funciona el sistema completo.

Bases de Datos para IA

Agenda del día

- Arquitecturas de datos.
- Computación en la nube.
- IaaS, PaaS y servicios gestionados.
- Infraestructura on-premise, cloud e híbrida.
- Sistemas transaccionales y analíticos.
- Data Warehouse.
- Modelado dimensional.
- Data Lake.
- Data Lakehouse.
- Arquitectura Medallion.
- Introducción a Big Data.
- Diseño de un pipeline Bronce → Plata → Oro.



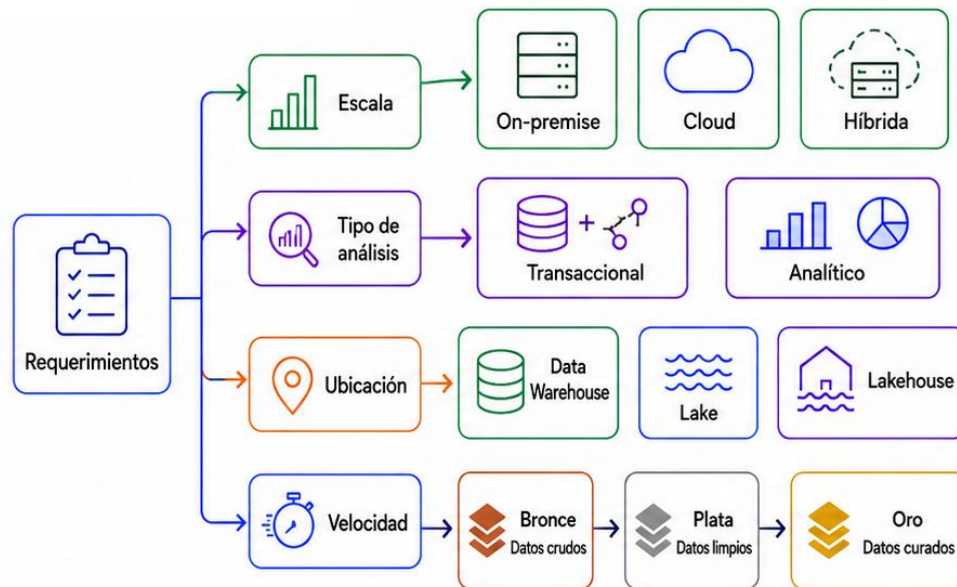
Hoy vamos a recorrer las principales arquitecturas de datos.
Veremos cómo se almacenan, procesan y organizan los datos en sistemas modernos.

Objetivos de la clase

Comprender cómo se organiza una infraestructura de datos

Al finalizar la clase, deberíamos poder:

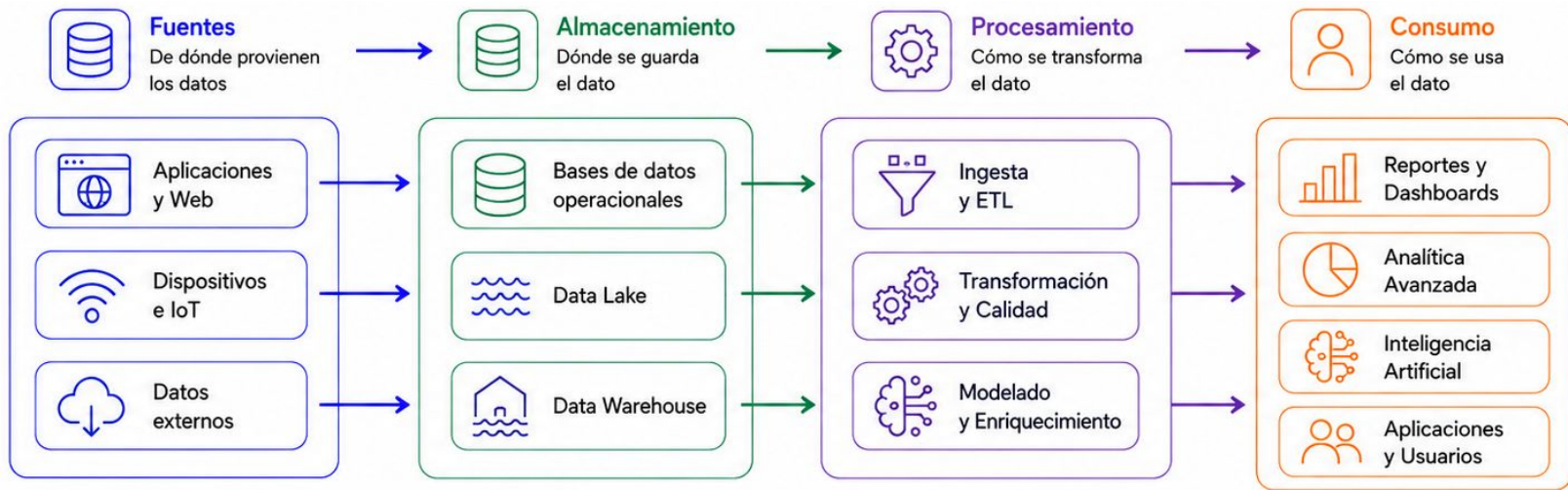
- reconocer los componentes principales de una arquitectura de datos;
- diferenciar infraestructura local, cloud e híbrida;
- distinguir cargas transaccionales de cargas analíticas;
- comprender Data Warehouse, Data Lake y Data Lakehouse;
- explicar la arquitectura Medallion;
- reconocer desafíos asociados a Big Data;
- diseñar un pipeline de datos por capas;
- seleccionar una arquitectura según un proyecto de IA.



El objetivo no es memorizar productos, sino comprender qué problema resuelve cada arquitectura.

De la Base a la Arquitectura

El recorrido completo del dato



De una base aislada a un sistema completo.
Ahora vamos a mirar todo el recorrido del dato.

Arquitectura de datos

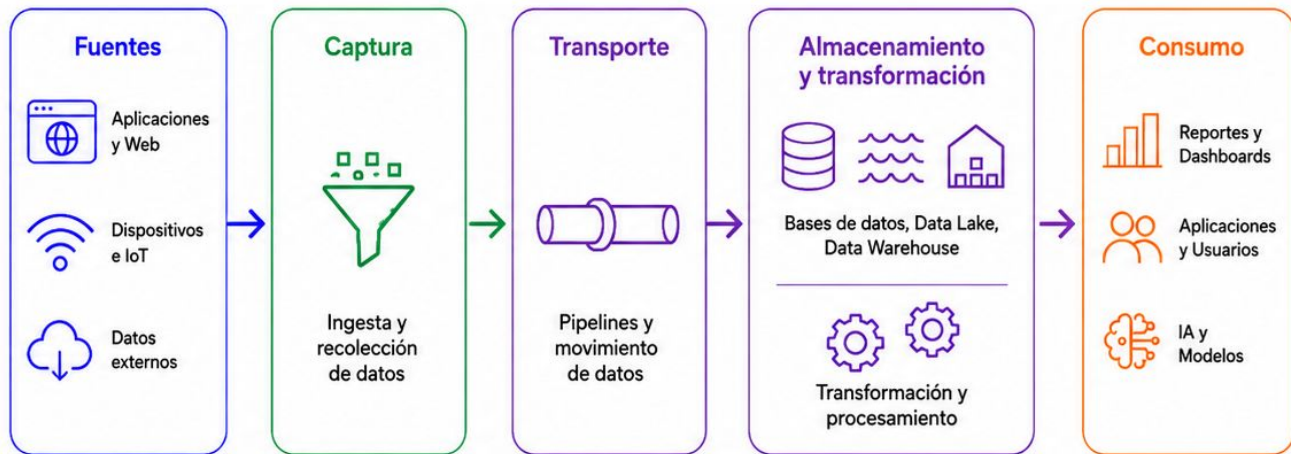
Más que almacenamiento

Una arquitectura de datos define cómo la información:

- se genera;
- se captura;
- se transporta;
- se almacena;
- se transforma;
- se protege;
- se consulta;
- se consume.

No describe solamente una base de datos.

Describe cómo se relacionan todos los componentes del sistema.



La arquitectura organiza el ciclo de vida completo del dato.

Elementos de una arquitectura

Del origen al consumo

Fuentes

- aplicaciones;
- archivos;
- APIs;
- sensores;
- logs.

Procesamiento

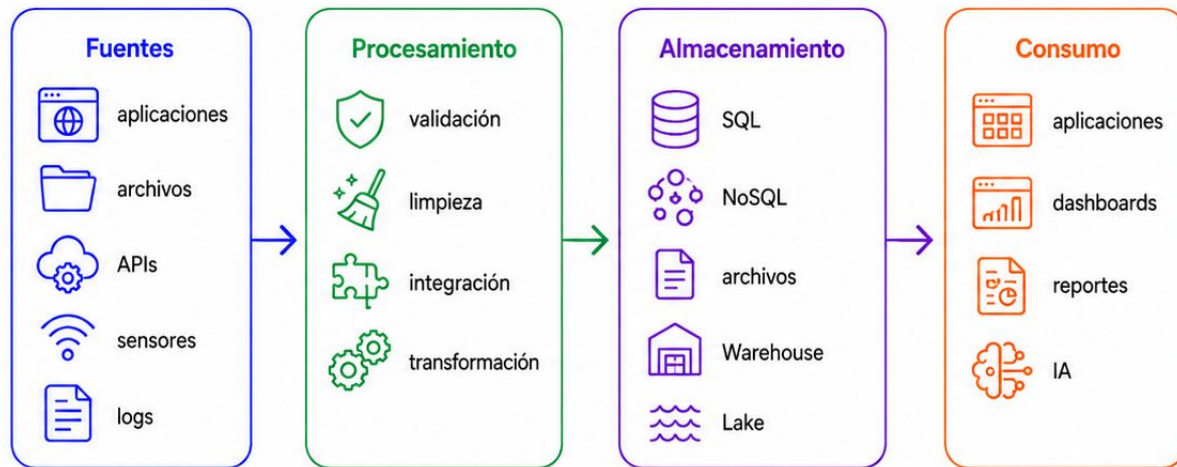
- validación;
- limpieza;
- integración;
- transformación.

Almacenamiento

- SQL;
- NoSQL;
- archivos;
- Warehouse;
- Lake.

Consumo

- aplicaciones;
- dashboards;
- reportes;
- modelos de IA.



Cada componente puede usar una tecnología distinta, pero todos forman parte del mismo sistema.

Caso guía

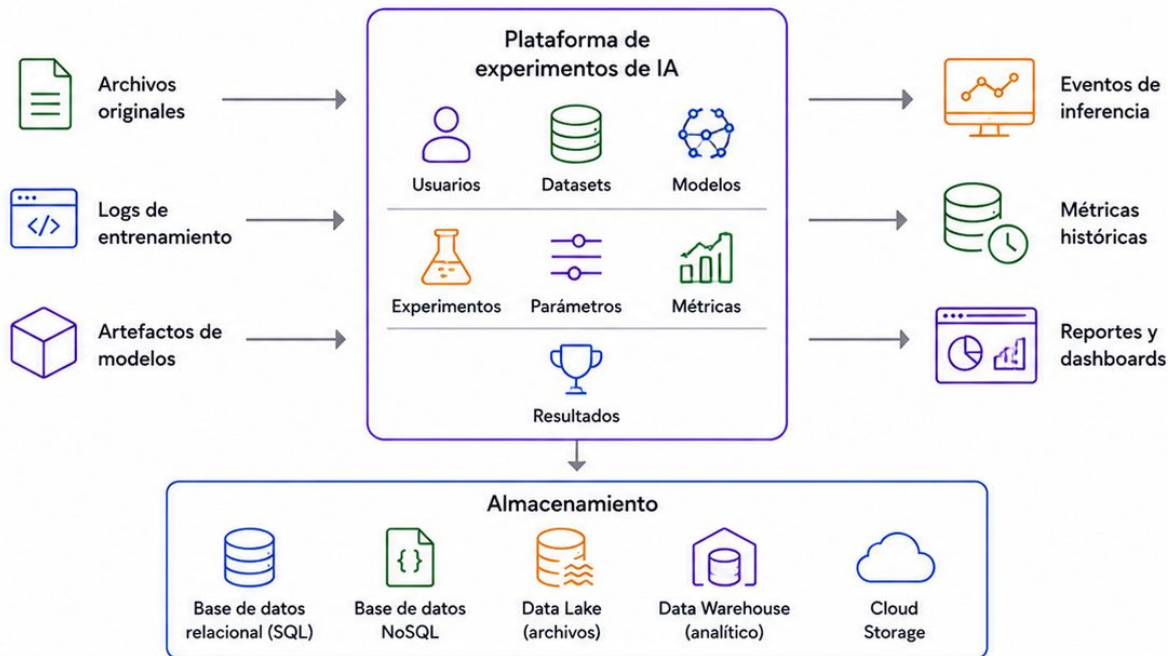
Plataforma de experimentos de IA

Hasta ahora trabajamos con:

- usuarios;
- datasets;
- modelos;
- experimentos;
- parámetros;
- métricas;
- resultados.

Ahora vamos a sumar:

- archivos originales;
- logs de entrenamiento;
- artefactos de modelos;
- eventos de inferencia;
- métricas históricas;
- reportes y dashboards.



El caso deja de ser una base aislada
y pasa a ser una plataforma de datos.

Antes de elegir tecnologías

Entender el recorrido del dato

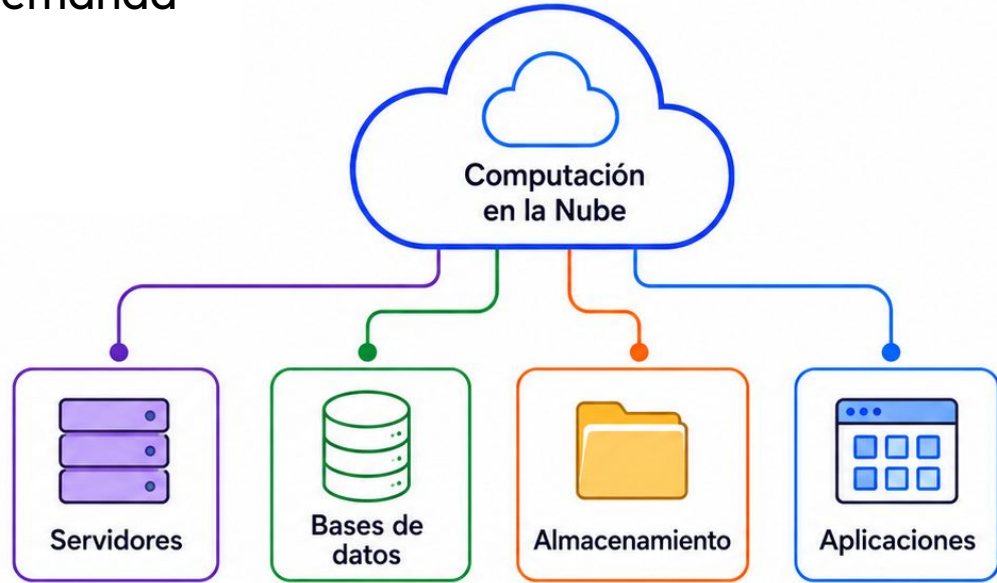
- ✓ ¿De dónde provienen los datos?
- ✓ ¿Con qué frecuencia se generan?
- ✓ ¿Qué volumen debemos almacenar?
- ✓ ¿Qué estructura tienen?
- ✓ ¿Debemos conservar el formato original?
- ✓ ¿Qué transformaciones necesitan?
- ✓ ¿Quién va a consumirlos?
- ✓ ¿Necesitamos información en tiempo real?
- ✓ ¿Qué nivel de seguridad requieren?
- ✓ ¿Dónde se ejecutará la infraestructura?



La arquitectura se diseña desde los requerimientos, no desde el catálogo de herramientas.

Computación en la Nube

Infraestructura y servicios bajo demanda



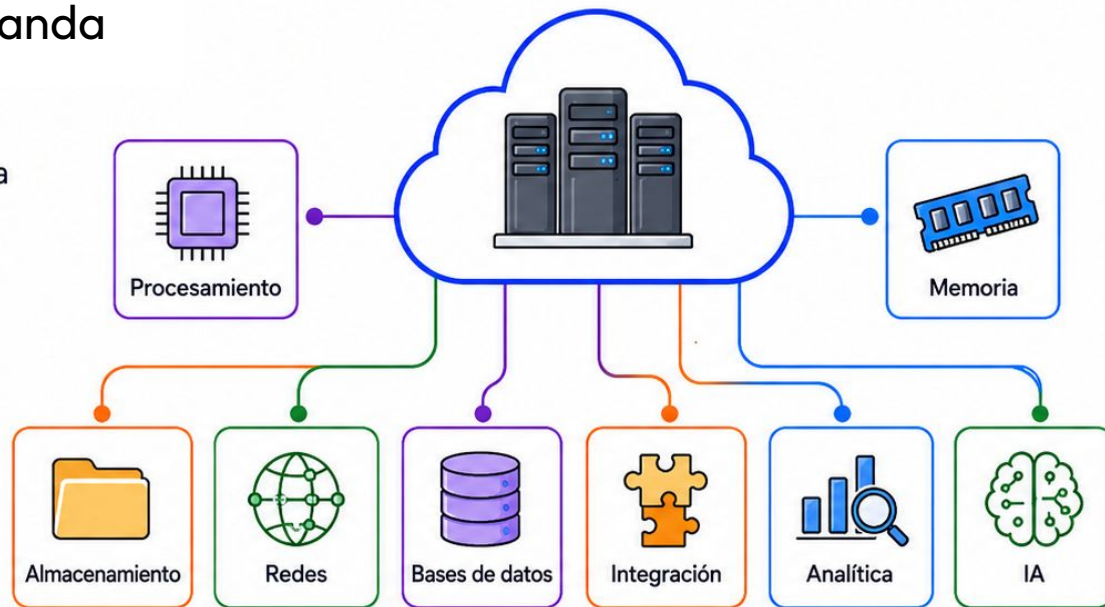
La nube permite acceder a infraestructura y servicios de datos bajo demanda.

Cloud computing

Recursos tecnológicos bajo demanda

La computación en la nube permite acceder a recursos informáticos a través de una plataforma externa:

- procesamiento;
- memoria;
- almacenamiento;
- redes;
- bases de datos;
- servicios de integración;
- herramientas analíticas;
- servicios de inteligencia artificial.



Los recursos pueden crearse, ampliarse o eliminarse según la necesidad.



Cloud permite consumir infraestructura como un servicio configurable.

De comprar a consumir

Infraestructura como servicio

En una infraestructura tradicional, la organización debe:

- comprar servidores;
- instalar sistemas operativos;
- configurar redes;
- administrar discos;
- mantener equipos;
- planificar capacidad;
- reemplazar componentes.

En cloud, parte de esas responsabilidades puede delegarse al proveedor.

Cambio de modelo

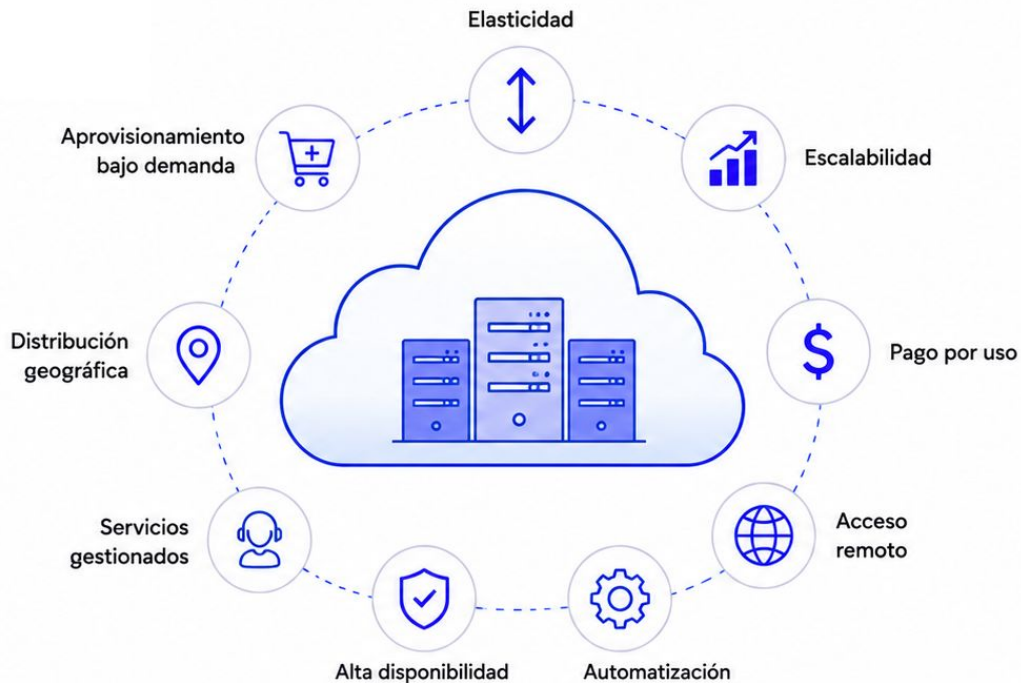


**Cloud no elimina la infraestructura.
Cambia quién la administra y cómo se utiliza.**

Características cloud

Flexibilidad y automatización

- Aprovisionamiento bajo demanda.
- Elasticidad.
- Escalabilidad.
- Pago por uso.
- Acceso remoto.
- Automatización.
- Alta disponibilidad.
- Servicios gestionados.
- Distribución geográfica.



La nube permite adaptar los recursos
al uso real del sistema.

Crece según la demanda

Dos conceptos relacionados



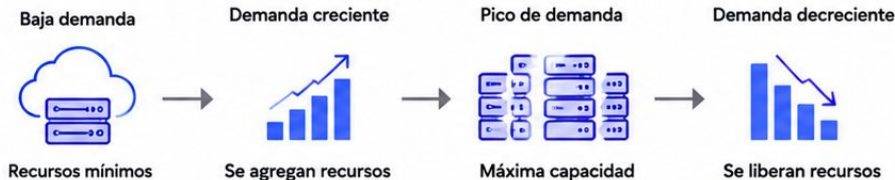
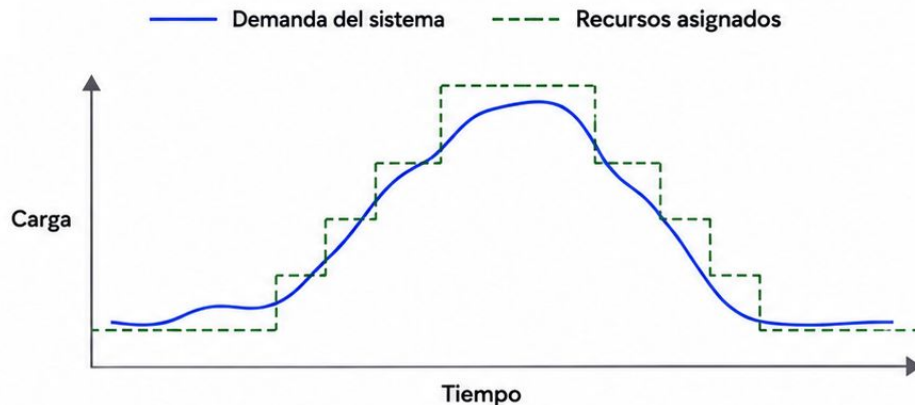
Escalabilidad es la capacidad de soportar una mayor carga agregando recursos.



Elasticidad es la capacidad de aumentar o reducir esos recursos dinámicamente.

Ejemplo:

- durante un entrenamiento intensivo se utilizan más recursos;
- al finalizar, la capacidad se reduce;
- el almacenamiento permanece disponible;
- el costo de cómputo disminuye.



Escalar es poder crecer.

Elasticidad es crecer y reducirse según la necesidad.

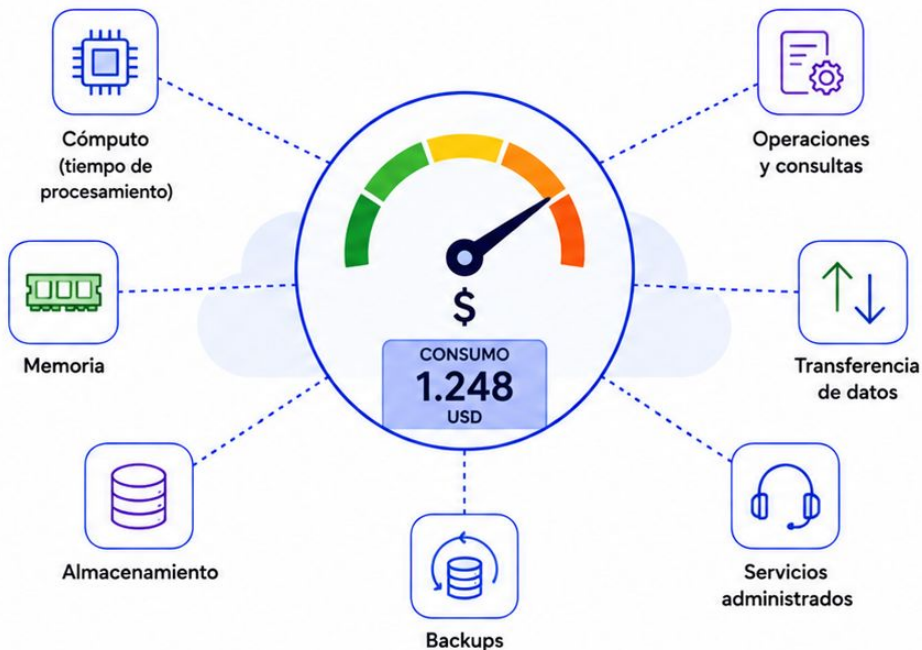
Modelo de costos

Pagar según el consumo

En cloud pueden facturarse:

- tiempo de procesamiento;
- memoria;
- almacenamiento;
- cantidad de operaciones;
- consultas;
- transferencia de datos;
- backups;
- servicios administrados.

Este modelo reduce la inversión inicial, pero no garantiza costos bajos.



Pago por uso no significa automáticamente pagar menos.

Responsabilidad compartida

Proveedor y organización



Cloud no transfiere automáticamente toda la responsabilidad de seguridad.

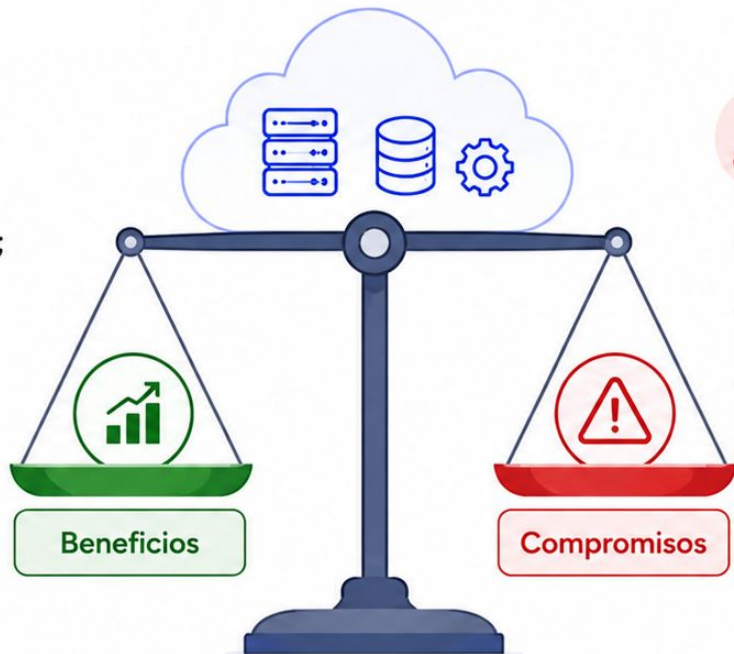
Adoptar cloud

Beneficios y nuevas decisiones



Cloud puede aportar:

- velocidad de implementación;
- escalabilidad;
- servicios gestionados;
- integración;
- disponibilidad;
- automatización.



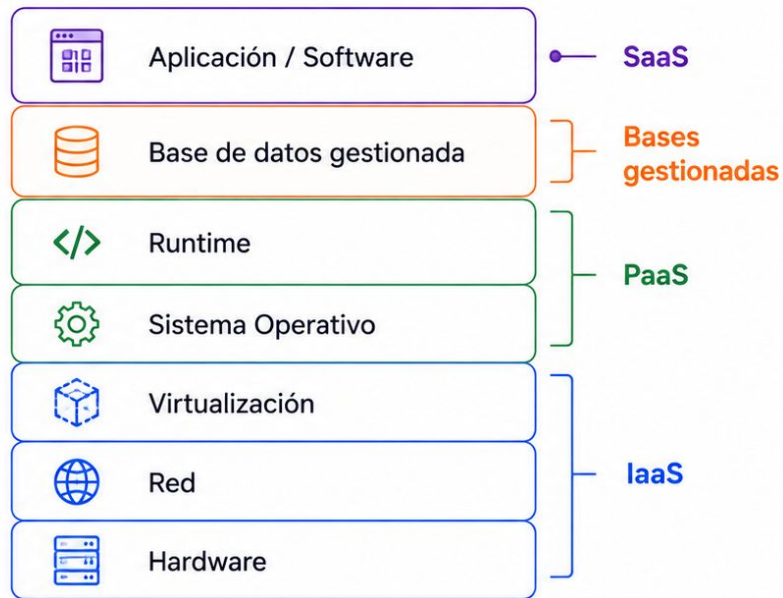
Pero también introduce:

- costos variables;
- dependencia tecnológica;
- complejidad de seguridad;
- costos de transferencia;
- necesidad de nuevos conocimientos.



Cloud simplifica algunas tareas, pero introduce otras responsabilidades.

Modelos de Servicio



Los modelos de servicio definen qué capa administra el proveedor y qué capa queda bajo control del usuario.

Niveles de servicio

Distintos niveles de control

IaaS

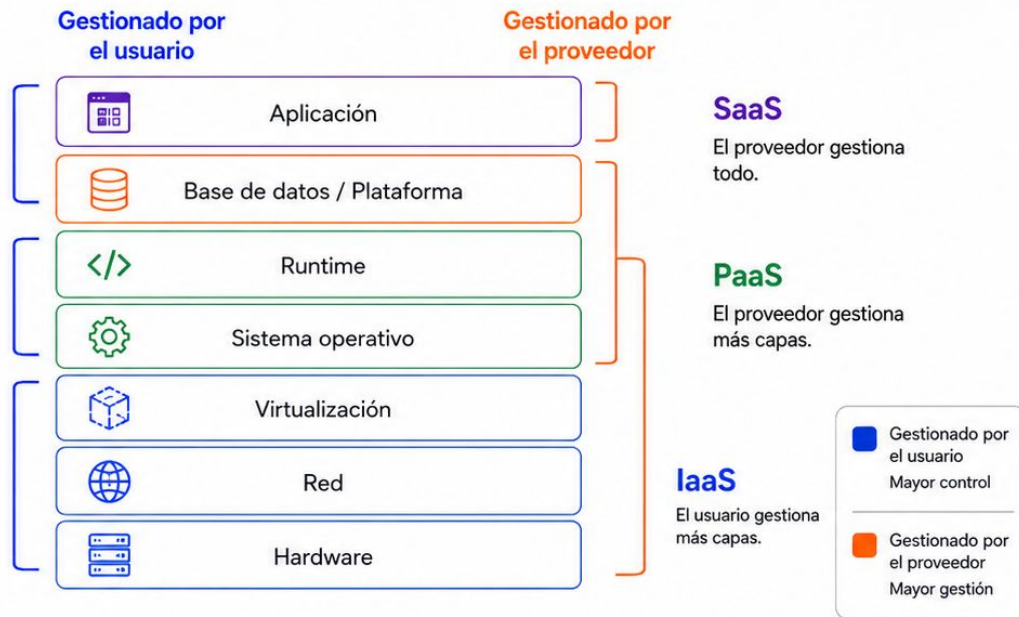
- máquinas virtuales;
- redes;
- discos;
- mayor control.

PaaS

- plataforma preparada;
- menos administración;
- mayor automatización.

SaaS

- aplicación completa;
- lista para usar;
- menor control técnico.



Cuanto más gestionado es el servicio, menos infraestructura administramos directamente.

IaaS

Infraestructura como servicio



En IaaS podemos crear:

- máquinas virtuales;
- discos;
- redes;
- balanceadores;
- firewalls;
- servidores de bases de datos.



La organización administra:

- sistema operativo;
- instalación;
- configuración;
- actualizaciones;
- backups;
- monitoreo.

Máquina virtual (VM)



Responsabilidad de la organización

Administra el sistema operativo, las aplicaciones y los datos.

Mayor control.

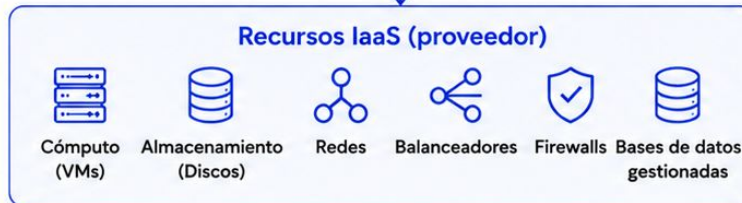


Responsabilidad del proveedor

Administra la infraestructura física y los servicios base.

Menor control.

Recursos IaaS (proveedor)



IaaS ofrece control, pero mantiene gran parte de la responsabilidad operativa.

PaaS

Plataforma lista para utilizar



En PaaS, el proveedor administra **gran parte de la infraestructura.**



La organización se concentra en:

- aplicaciones;
- configuraciones;
- modelos de datos;
- reglas de negocio;
- integraciones;
- consumo del servicio.



PaaS reduce tareas operativas y acelera la implementación.

Database as a Service

Delegar la operación técnica



Una base gestionada puede incluir:

- aprovisionamiento;
- actualizaciones;
- parches;
- monitoreo;
- backups;
- recuperación;
- replicación;
- alta disponibilidad;
- escalado.

La organización sigue diseñando el modelo y las consultas.

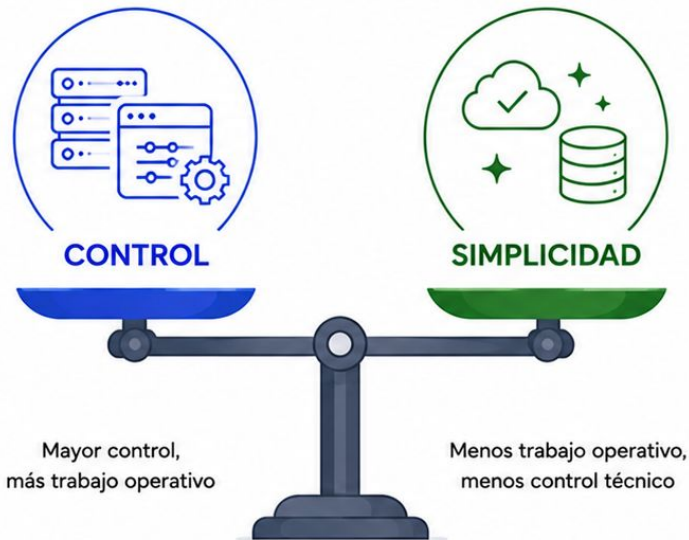


El proveedor administra la plataforma.
El equipo sigue siendo responsable del buen diseño.

Comparación

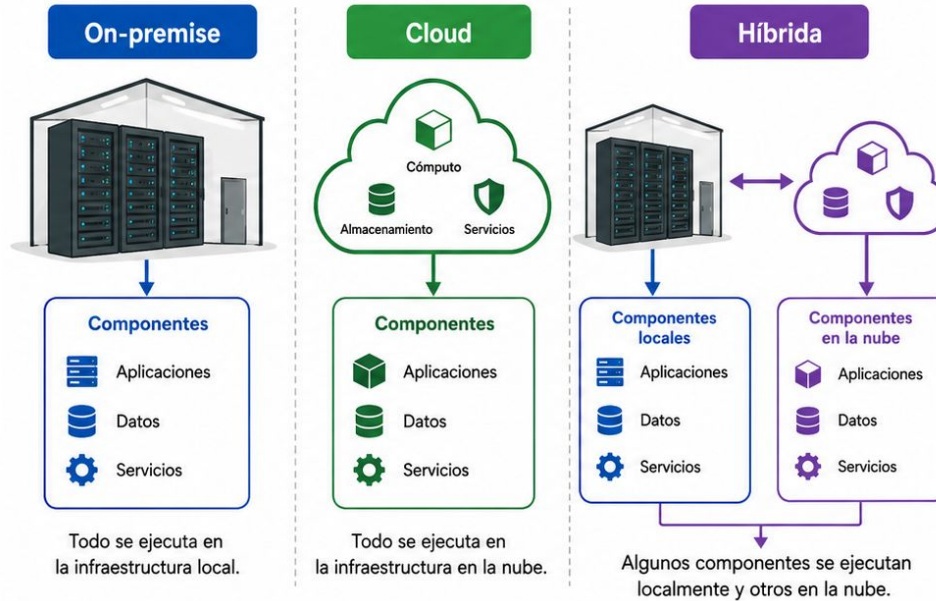
Control frente a simplicidad

Aspecto	IaaS	Servicio gestionado
 Instalación	Manual	Automatizada
 Sistema operativo	Equipo	Proveedor
 Actualizaciones	Equipo	Generalmente proveedor
 Backups	Configurables	Integrados
 Control	Alto	Compartido
 Operación	Compleja	Simplificada
 Personalización	Mayor	Más limitada



La elección depende del control requerido y de la capacidad operativa del equipo.

On-Premise, Cloud e Híbrida



La infraestructura puede ejecutarse localmente, en la nube o en una combinación de ambas.

On-premise

Infraestructura dentro de la organización



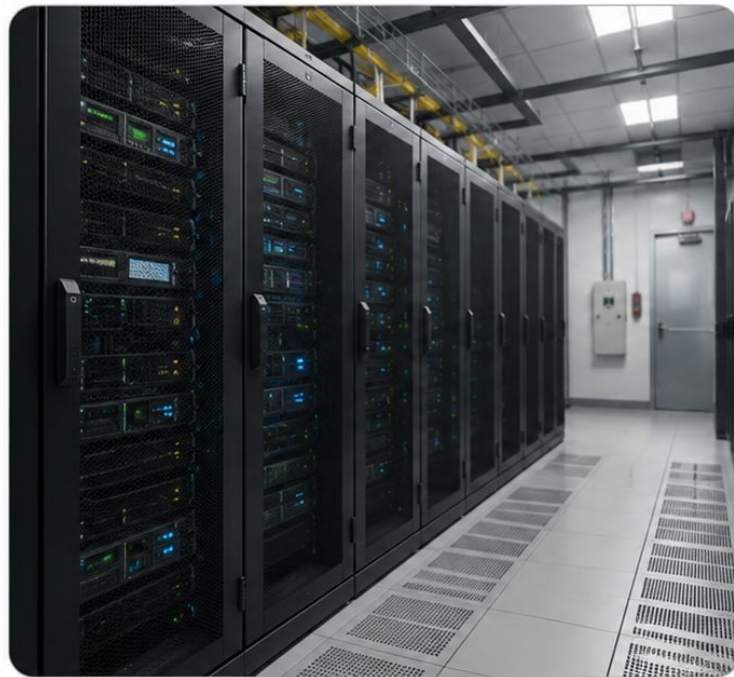
En una arquitectura on-premise, la organización administra:

- servidores;
- redes;
- almacenamiento;
- energía;
- seguridad física;
- software;
- backups;
- monitoreo.



Puede resultar conveniente cuando:

- se necesita control directo;
- la latencia local es crítica;
- los datos no pueden salir;
- existen sistemas heredados.



On-premise ofrece control, pero exige capacidad técnica y operativa.

Infraestructura propia

Control frente a capacidad de crecimiento



Ventajas



control físico;



personalización;



baja latencia local;



integración con sistemas existentes;



costos previsible en
cargas estables.



Limitaciones



inversión inicial;



escalado lento;



mantenimiento permanente;



obsolescencia;



recuperación ante desastres
más compleja.



El control también implica asumir toda la operación.

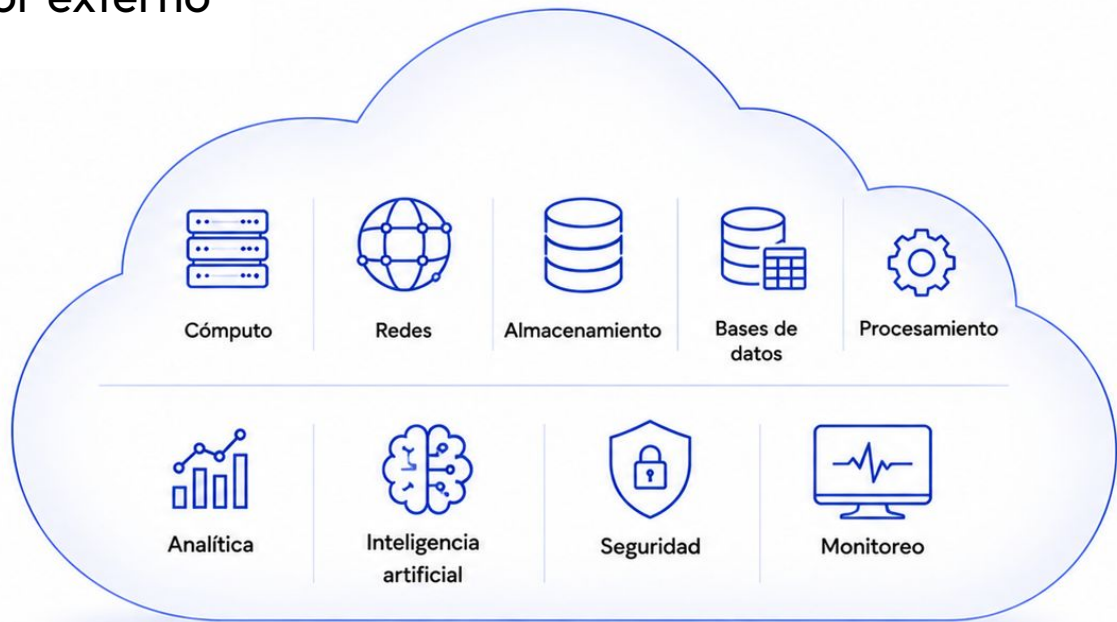
Cloud

Infraestructura en un proveedor externo



Una arquitectura cloud utiliza servicios remotos para:

- cómputo;
- redes;
- almacenamiento;
- bases de datos;
- procesamiento;
- analítica;
- inteligencia artificial;
- seguridad;
- monitoreo.



Cloud permite crecer y desplegar más rápido,
pero requiere una nueva forma de diseñar y operar.

Arquitectura cloud

Flexibilidad con nuevos compromisos



Ventajas



aprovisionamiento rápido;



elasticidad;



escalabilidad;



alta disponibilidad;



servicios gestionados;



integración con IA.



Limitaciones



costos variables;



dependencia de conectividad;



vendor lock-in;



complejidad de seguridad;



costos de transferencia.



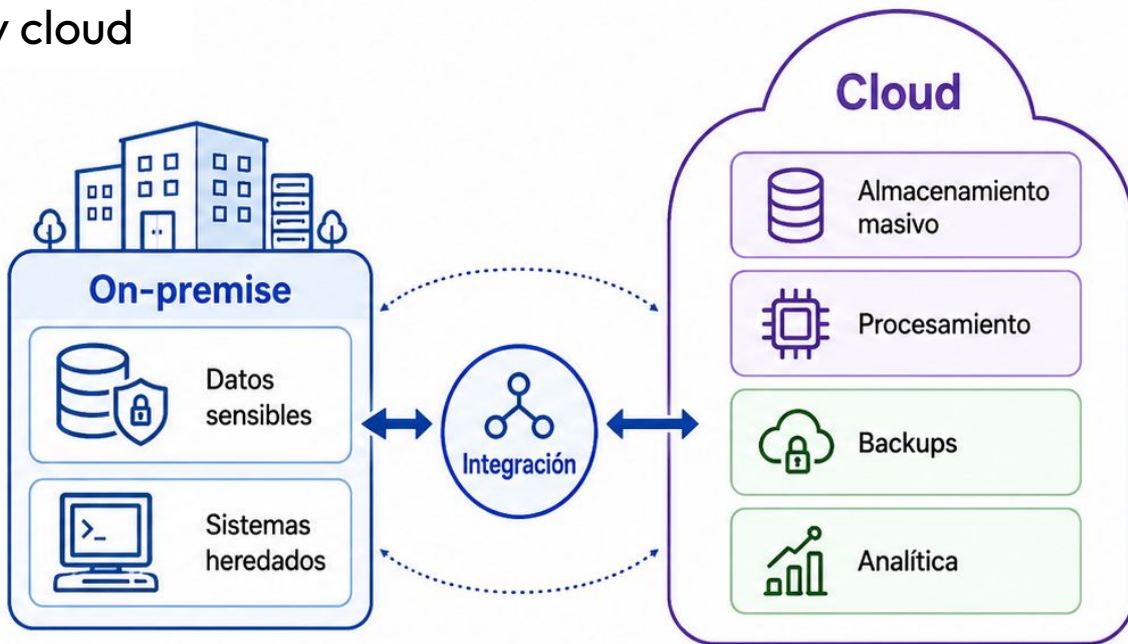
Cloud reduce barreras de entrada,
pero no elimina las decisiones de arquitectura.

Arquitectura híbrida

Combinar infraestructura local y cloud

Una arquitectura híbrida puede mantener:

- datos sensibles localmente;
- sistemas heredados on-premise;
- almacenamiento masivo en cloud;
- procesamiento intensivo en cloud;
- backups remotos;
- servicios analíticos externos.



**Híbrido no es tener dos infraestructuras separadas.
Es diseñarlas para trabajar juntas.**

On-premise, cloud o híbrida

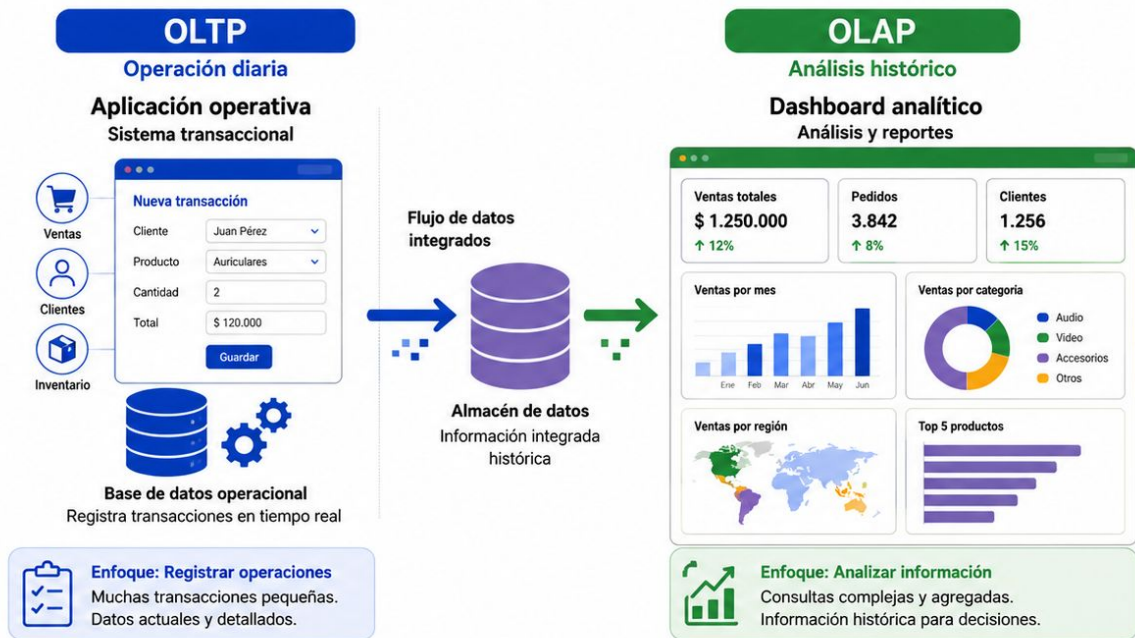
Preguntas para decidir

- ¿Los datos pueden salir de la organización?
- ¿La latencia local es crítica?
- ¿La carga es estable o variable?
- ¿Tenemos personal para operar infraestructura?
- ¿Necesitamos escalar rápidamente?
- ¿Qué exige la normativa?
- ¿Qué presupuesto inicial existe?
- ¿Qué nivel de disponibilidad necesitamos?



La ubicación de la infraestructura también es una decisión de arquitectura.

OLTP y OLAP



OLTP registra la operación cotidiana.

OLAP permite analizar la información para tomar decisiones.

Operación y análisis

No todas las consultas tienen el mismo propósito



Una aplicación operativa necesita:

- insertar;
- actualizar;
- validar;
- responder rápido;
- mantener consistencia.



Un sistema analítico necesita:

- recorrer muchos registros;
- agrupar;
- comparar períodos;
- calcular indicadores;
- analizar históricos.

Carga operativa



transacciones
pequeñas



respuesta
rápida

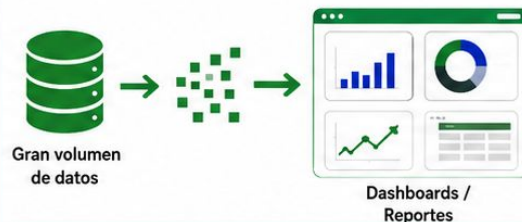


datos
actuales



Enfoque: operaciones en tiempo real.
Consistencia y rendimiento inmediato.

Carga analítica



muchos
registros



agregaciones



histórico



Enfoque: análisis y toma de decisiones.
Agregaciones e información histórica.



La misma base no siempre es la mejor opción para ambas cargas.

OLTP

Procesamiento transaccional en línea



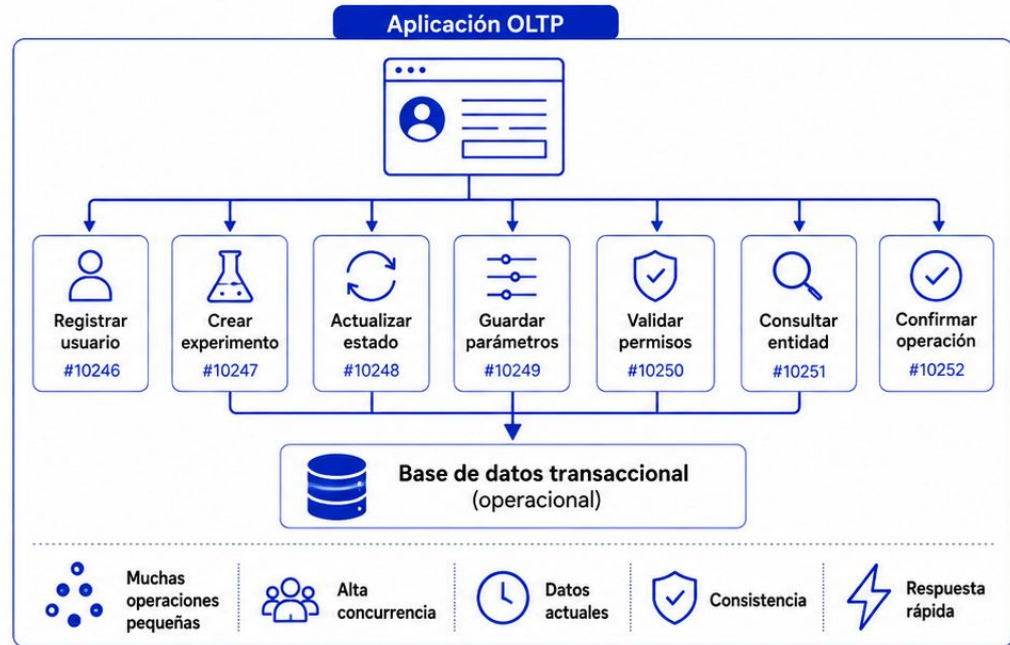
OLTP se orienta a operaciones frecuentes y breves:

- registrar usuarios;
- crear experimentos;
- actualizar estados;
- guardar parámetros;
- consultar una entidad;
- validar permisos;
- confirmar una operación.



Características:

- muchas operaciones pequeñas;
- alta concurrencia;
- datos actuales;
- consistencia;
- respuesta rápida.



OLTP sostiene la operación cotidiana del sistema.

OLAP

Procesamiento analítico en línea



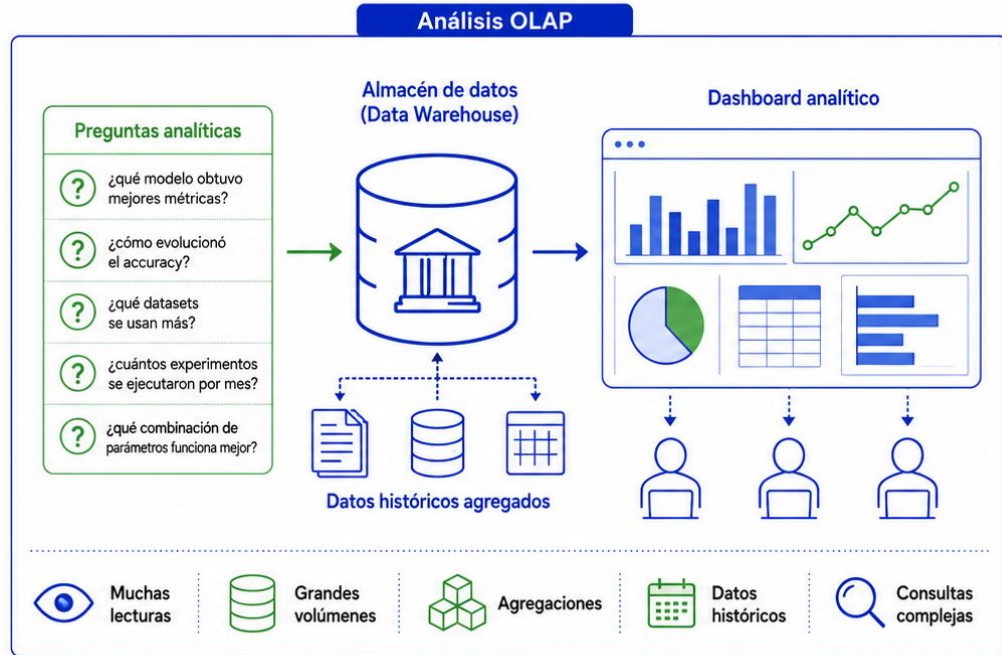
OLAP se orienta a preguntas como:

- ¿qué modelo obtuvo mejores métricas?;
- ¿cómo evolucionó el accuracy?;
- ¿qué datasets se usan más?;
- ¿cuántos experimentos se ejecutaron por mes?;
- ¿qué combinación de parámetros funciona mejor?



Características:

- muchas lecturas;
- grandes volúmenes;
- agregaciones;
- datos históricos;
- consultas complejas.



OLAP transforma datos acumulados en indicadores y conocimiento.

Comparación

Dos cargas diferentes

Aspecto	OLTP	OLAP
 Objetivo	Operación	Análisis
 Consultas	Cortas	Complejas
 Datos	Actuales	Históricos
 Operaciones	Insertar y actualizar	Leer y argupar
 Volumen por consulta	Bajo	Alto
 Modelo	Normalizado	Analítico
 Usuarios	Aplicaciones	Analistas y modelos

OLTP

Operacional /
Transaccional



- ✓ Procesa transacciones del día a día
- ✓ Datos actuales
- ✓ Enfocado en eficiencia

OLAP

Analítico /
Histórico



- ✓ Analiza grandes volúmenes de datos históricos
- ✓ Vista consolidada
- ✓ Enfocado en conocimiento



Separar cargas evita que el análisis afecte la operación.

Separación operativa y analítica

Evitar competencia por recursos

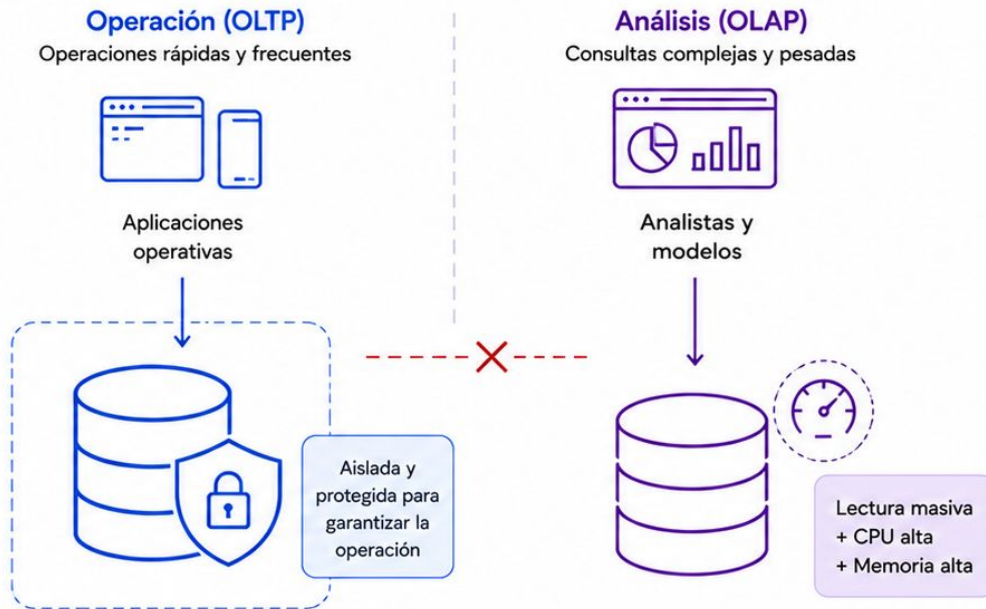


Una consulta analítica puede:

- leer millones de filas;
- consumir CPU;
- usar mucha memoria;
- afectar tiempos de respuesta;
- competir con operaciones críticas;
- bloquear recursos.

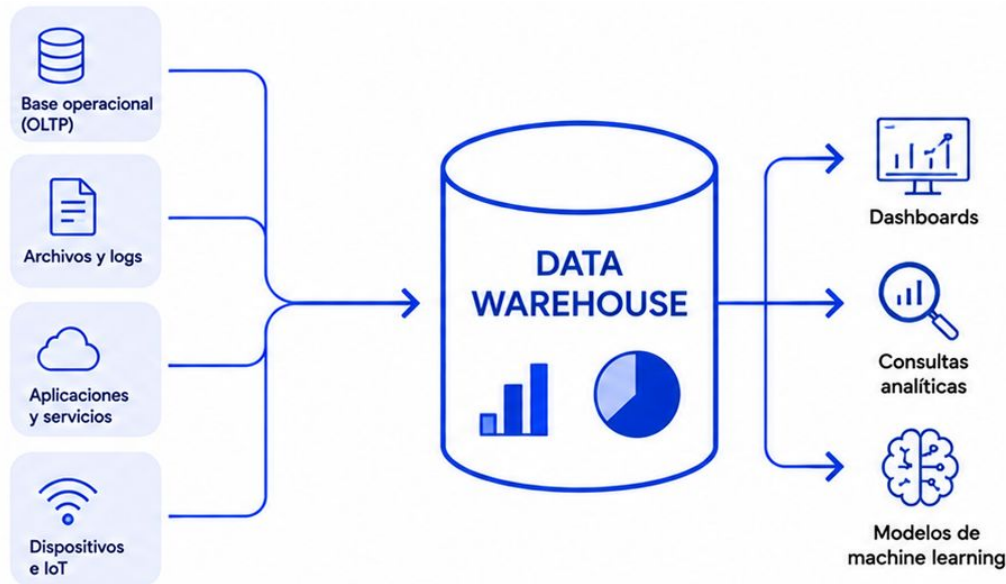


Por eso, muchas arquitecturas separan la base operativa del entorno analítico.



La base transaccional sostiene la operación.
La arquitectura analítica sostiene las preguntas.

Data Warehouse



El Data Warehouse integra múltiples fuentes
y prepara los datos para responder preguntas de negocio.

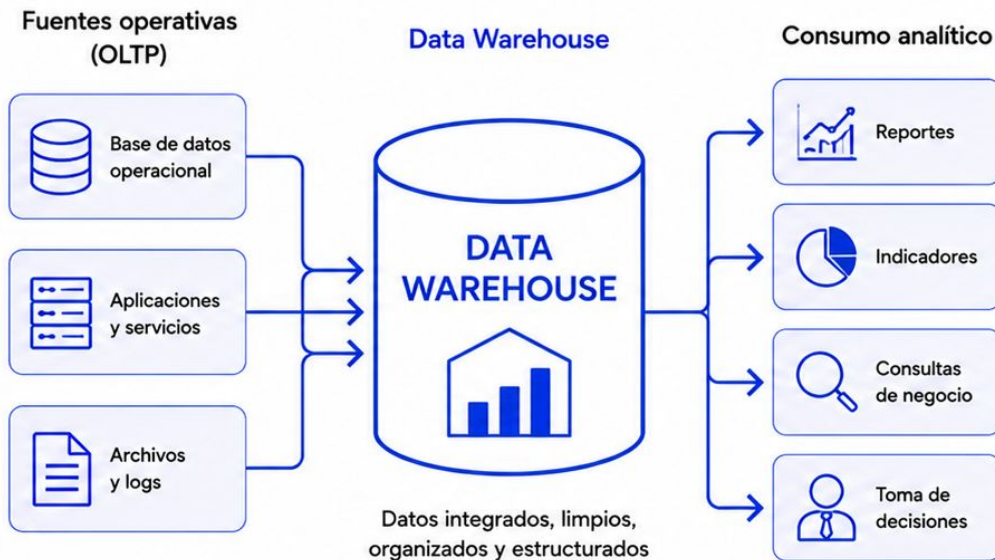
¿Qué es?

Repositorio analítico centralizado

Un Data Warehouse integra información de distintas fuentes para facilitar:

- análisis histórico;
- reportes;
- indicadores;
- comparaciones;
- consultas de negocio;
- toma de decisiones.

Los datos suelen estar limpios, organizados y estructurados.



El Warehouse no busca registrar la operación, sino facilitar su análisis.

Características

Datos preparados para consultar



- Datos estructurados.



- Integración de múltiples fuentes.



- Información histórica.



- Modelo analítico.



- Calidad controlada.



- Consultas complejas.



- Agregaciones.



- Orientación al negocio.

Múltiples fuentes

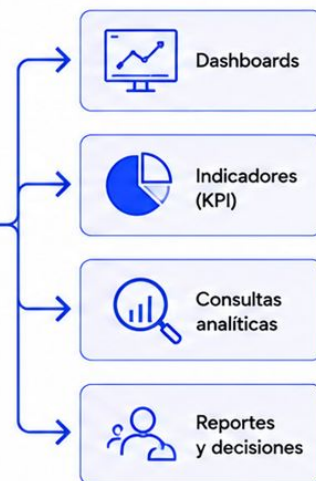


Data Warehouse



Datos integrados, limpios, organizados y estructurados para análisis.

Consumo analítico



El Data Warehouse busca construir una visión integrada y confiable.

Una visión común

Unificar información dispersa

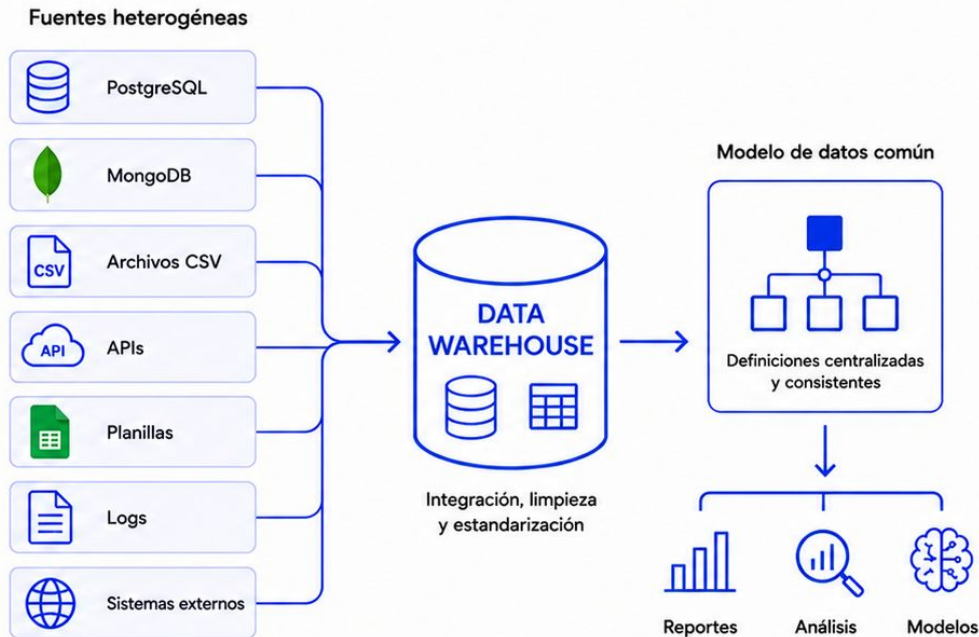
Una organización puede tener datos en:

- PostgreSQL;
- MongoDB;
- archivos CSV;
- APIs;
- planillas;
- logs;
- sistemas externos.

El Warehouse integra esas fuentes en una estructura común y centraliza definiciones.

Ejemplos:

- qué significa “experimento finalizado”;
- cómo se calcula el accuracy promedio;
- qué versión del modelo es la oficial.



Una fuente única de verdad busca que todos calculen lo mismo de la misma manera.

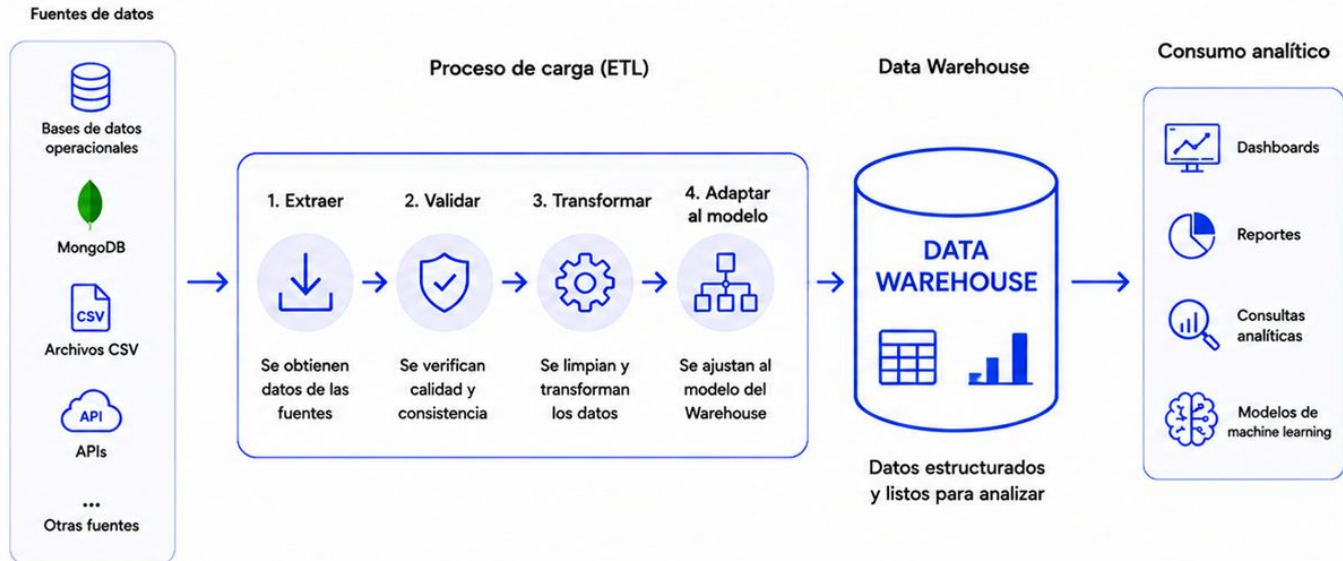
Esquema en escritura

Definir la estructura antes de almacenar

En un Data Warehouse, los datos suelen:

- extraerse de las fuentes;
- validarse;
- transformarse;
- adaptarse al modelo;
- almacenarse.

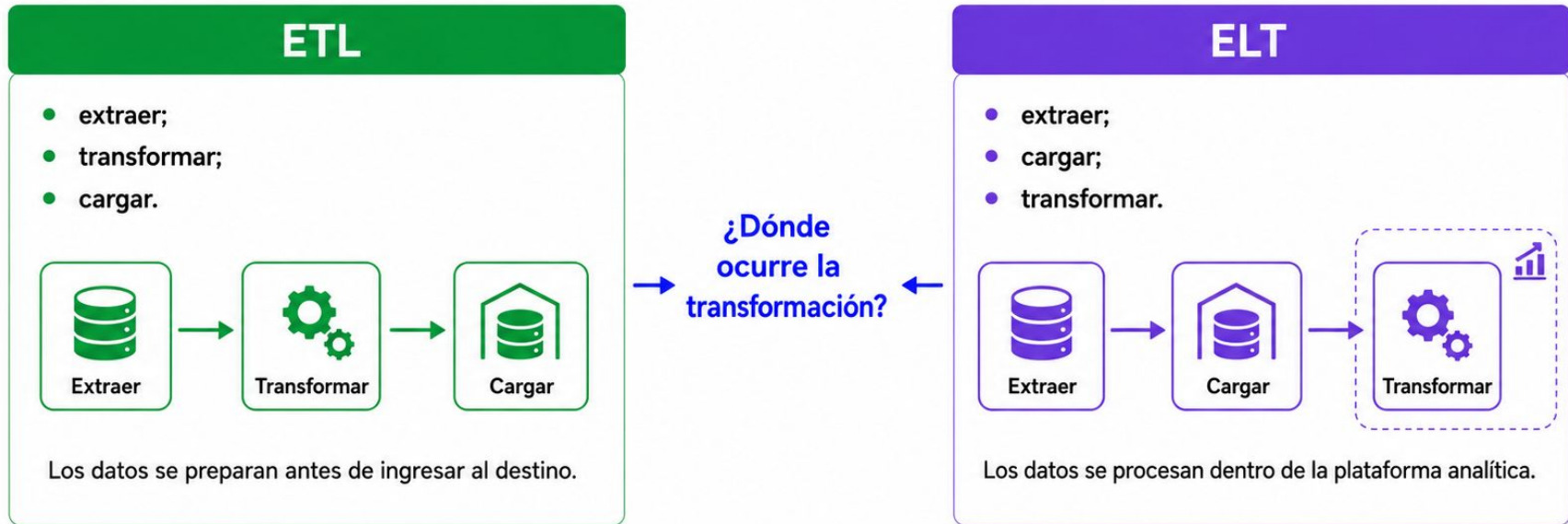
La estructura se define antes del consumo.



En el Warehouse, el dato debe ajustarse al esquema antes de quedar disponible.

ETL y ELT

Dos formas de organizar las transformaciones



ETL transforma antes de cargar.
ELT carga antes de transformar.

Modelo dimensional

Organizar datos para análisis

El modelado dimensional organiza la información alrededor de:

- hechos;
- dimensiones;
- medidas;
- jerarquías;
- períodos;
- categorías.

Busca simplificar las consultas y mejorar el rendimiento analítico.



El diseño analítico responde a preguntas de negocio,
no a operaciones transaccionales.

Hechos y dimensiones

Medidas y contexto

● Hechos

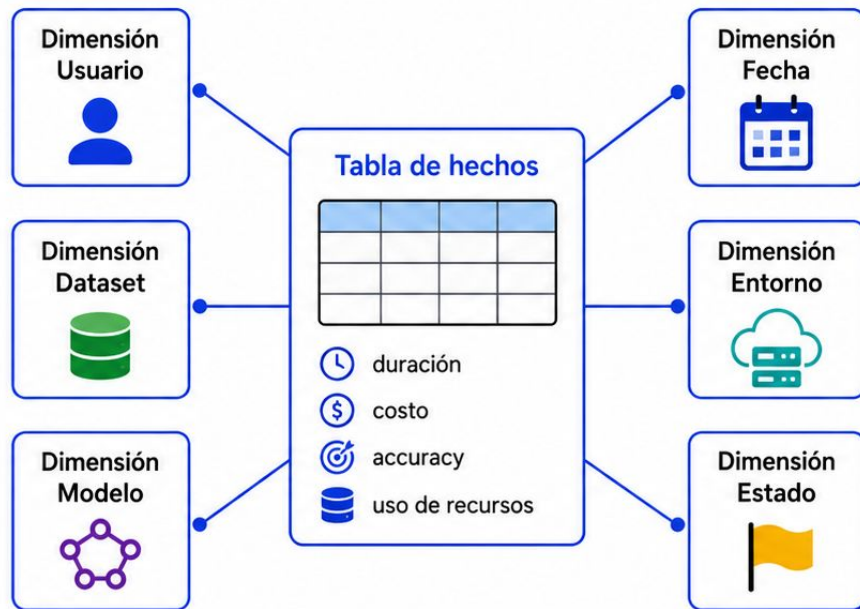
Representan eventos medibles:

- ejecución de un experimento;
- duración;
- costo;
- accuracy;
- uso de recursos.

● Dimensiones

Aportan contexto:

- usuario;
- dataset;
- modelo;
- fecha;
- entorno;
- estado.



Los hechos indican qué ocurrió.

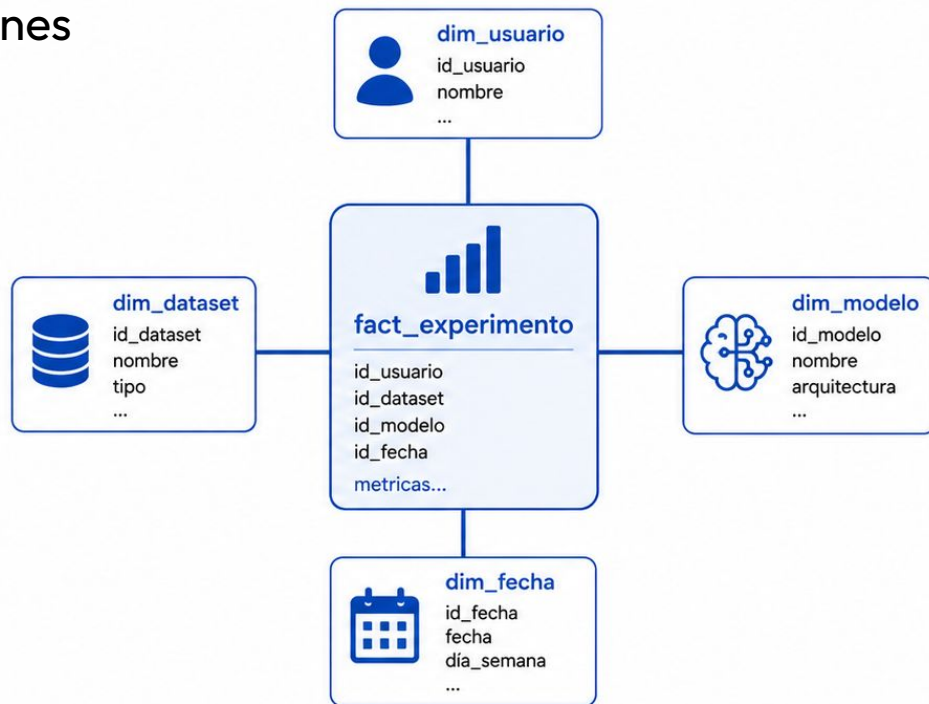
Las dimensiones permiten analizarlo desde distintas perspectivas.

Esquema estrella

Una tabla de hechos y varias dimensiones

En un esquema estrella:

- la tabla de hechos se ubica en el centro;
- las dimensiones se conectan directamente;
- las consultas son más simples;
- existen menos JOINS;
- puede haber redundancia en las dimensiones.



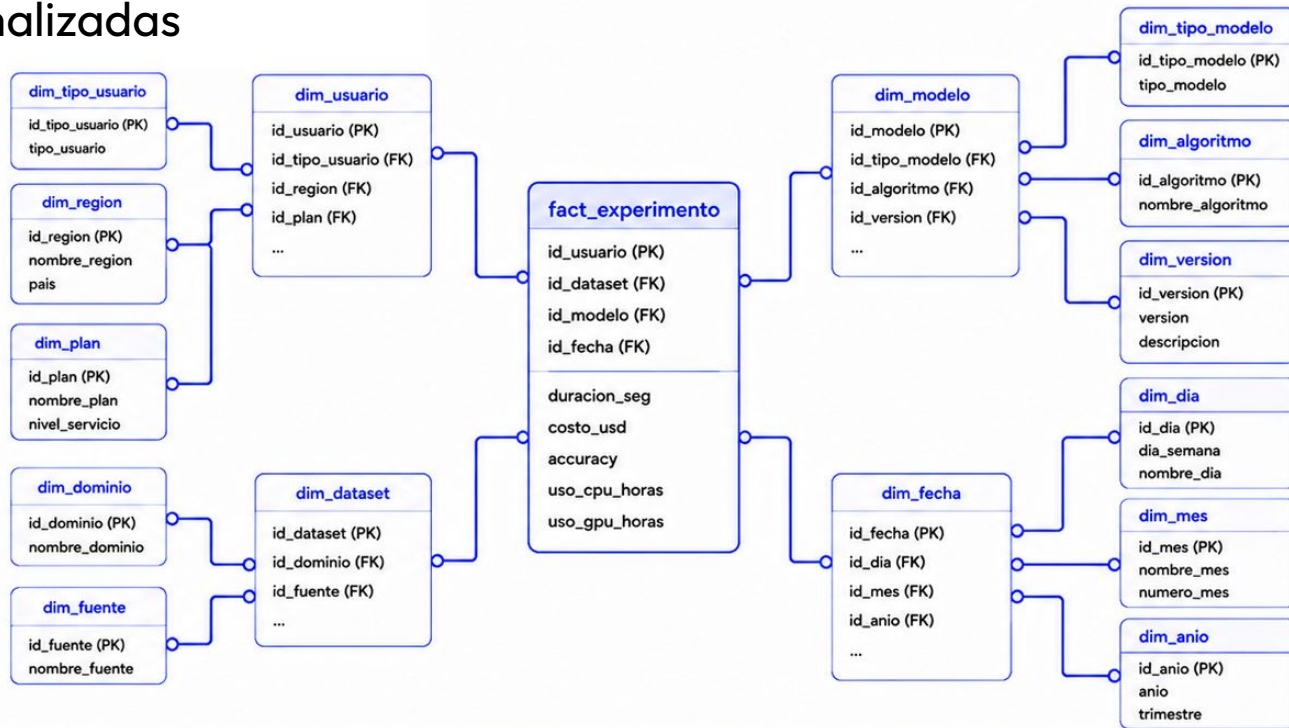
El esquema estrella prioriza simplicidad y rendimiento analítico.

Esquema copo de nieve

Dimensiones más normalizadas

En un esquema copo de nieve:

- las dimensiones se separan en más tablas;
- se reduce redundancia;
- aumenta la cantidad de relaciones;
- las consultas son más complejas;
- puede mejorar la consistencia de algunos atributos.



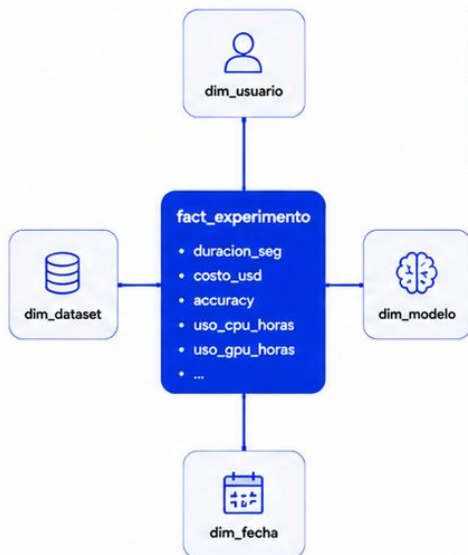
El copo de nieve normaliza más, pero exige más JOINS.

Comparación

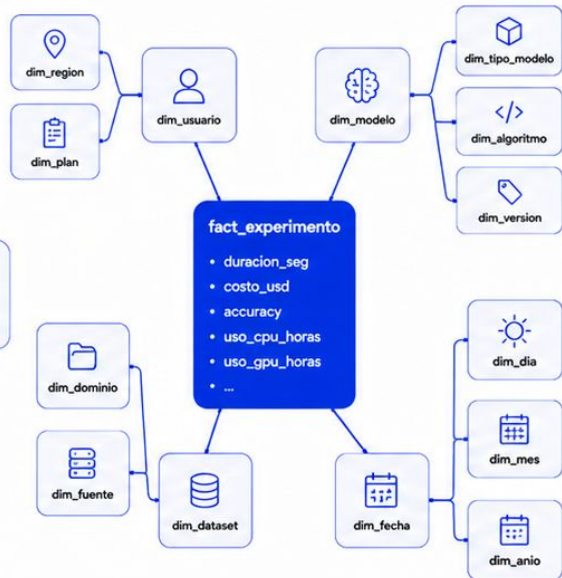
Simplicidad o mayor normalización

Aspecto	Estrella	Copo de nieve
 Dimensiones	Desnormalizadas	Normalizadas
 JOINS	Menos	Más
 Consultas	Simples	Más complejas
 Redundancia	Mayor	Menor
 Rendimiento	Generalmente mejor	Puede ser menor
 Mantenimiento	Simple	Más estructurado

Esquema estrella



Esquema copo de nieve

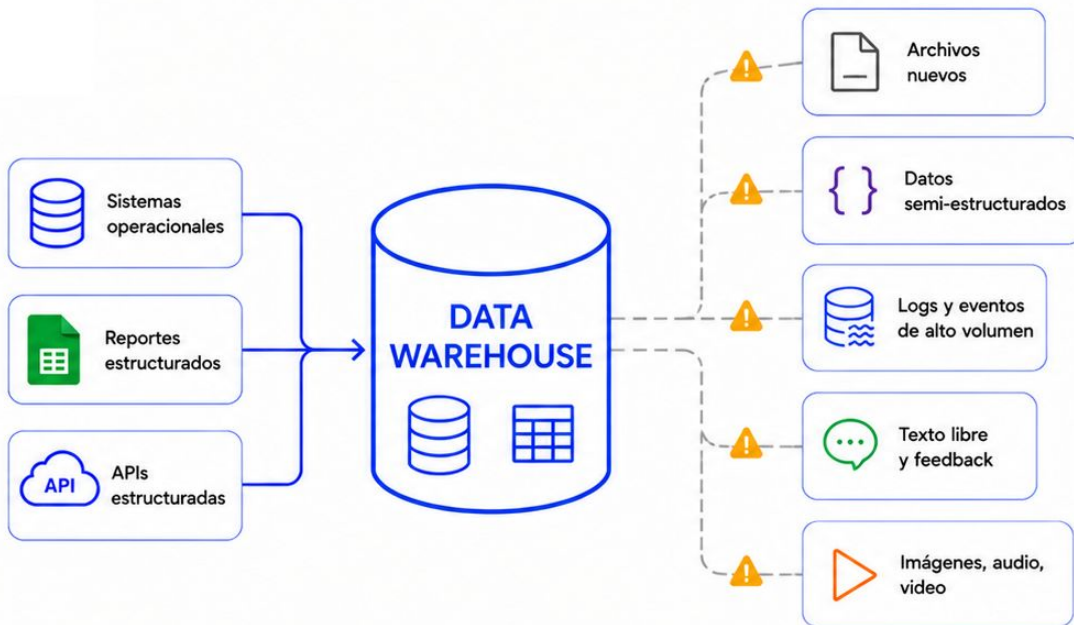


La elección depende del equilibrio entre simplicidad, rendimiento e integridad.

Limitaciones del Warehouse

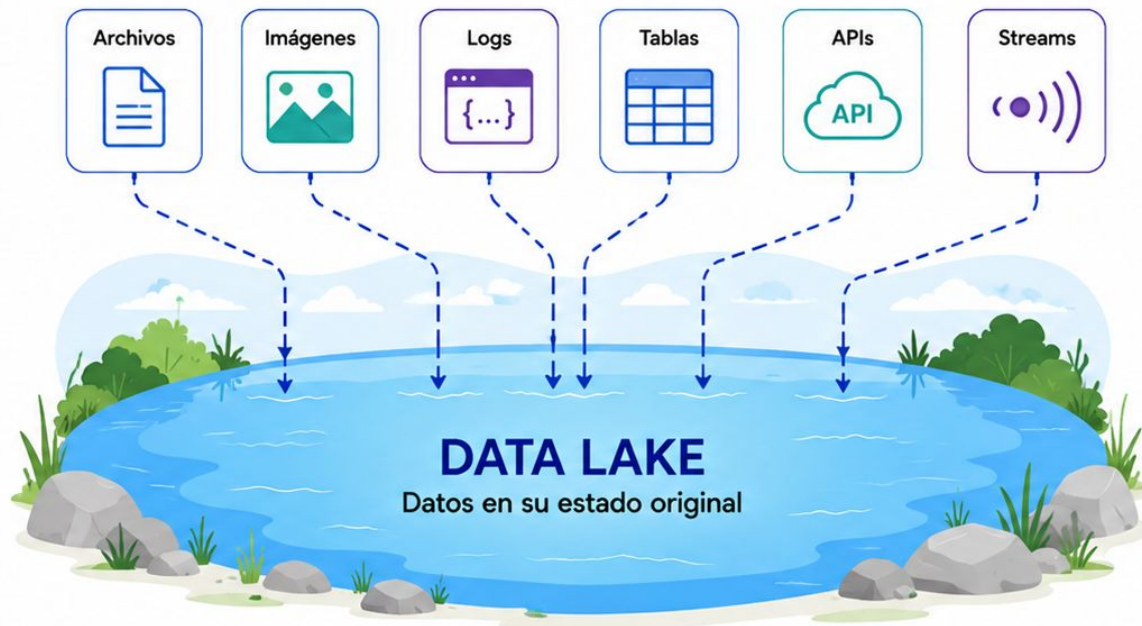
No resuelve todos los escenarios

- Requiere modelado previo.
- Incorporar nuevas fuentes puede ser lento.
- Trabaja mejor con datos estructurados.
- Puede ser costoso.
- No siempre es ideal para exploración.
- Puede limitar datos no estructurados.
- Exige procesos de integración definidos.



El Warehouse es muy potente para análisis estructurado, pero menos flexible ante datos cambiantes.

Data Lake



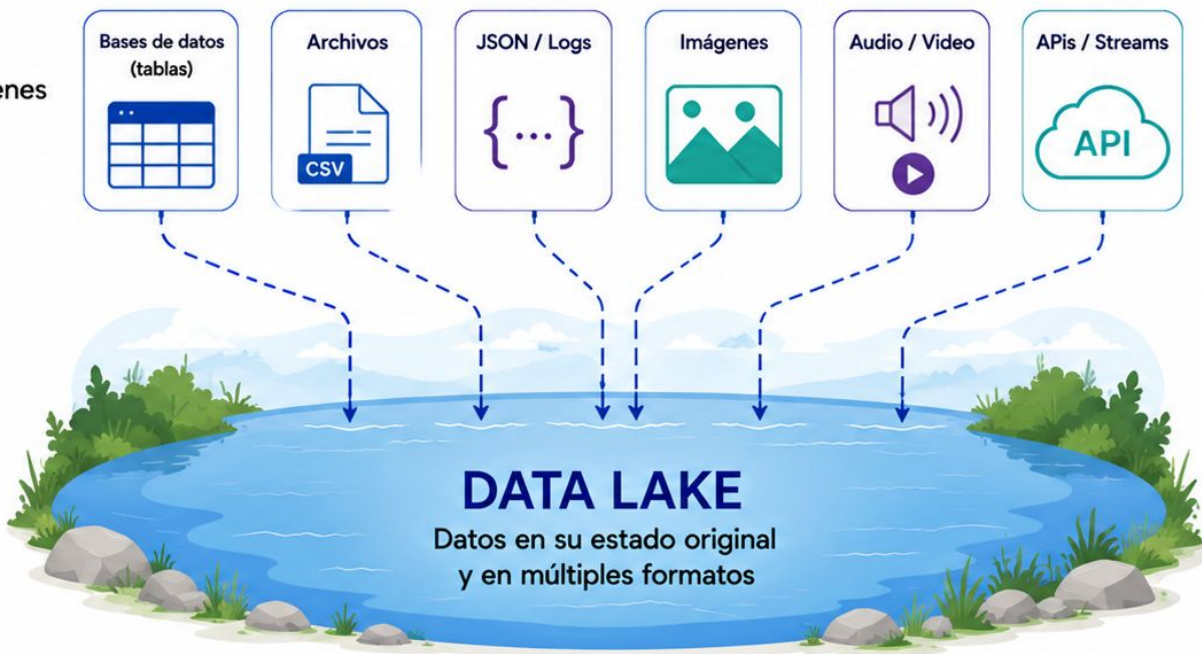
El Data Lake permite guardar todo tipo de datos, tal como se generan, para explorarlos y analizarlos cuando sea necesario.

¿Qué es?

Repositorio de datos crudos

Un Data Lake permite almacenar grandes volúmenes de datos:

- estructurados;
- semiestructurados;
- no estructurados;
- en distintos formatos;
- provenientes de múltiples fuentes;
- sin definir previamente todos sus usos.



El Data Lake conserva los datos con mayor flexibilidad.

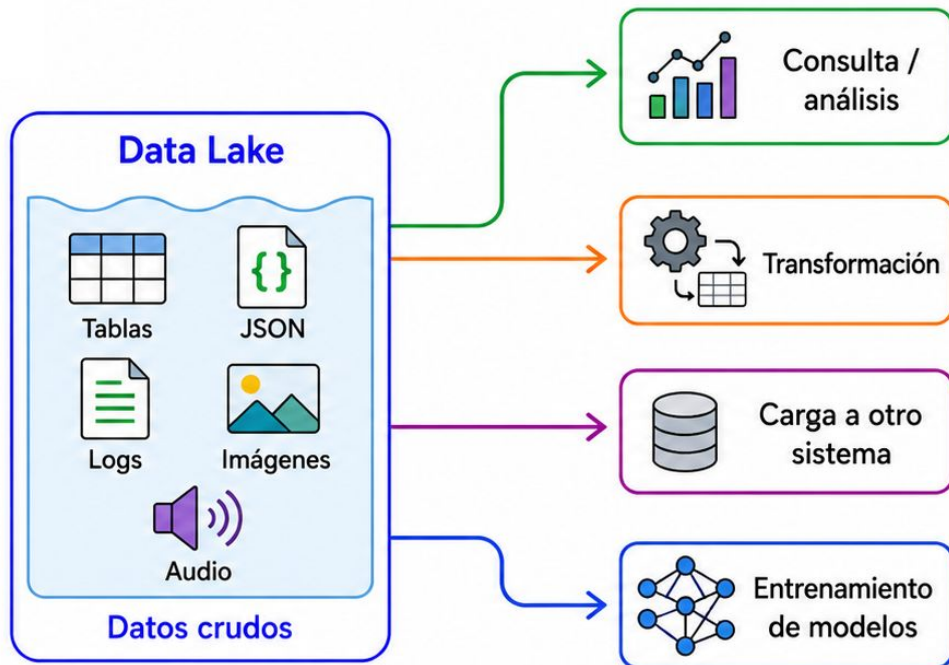
Esquema en lectura

Interpretar la estructura al consumir

En un Data Lake, los datos pueden almacenarse primero y estructurarse después.

El esquema se aplica cuando:

- se consultan;
- se transforman;
- se analizan;
- se cargan en otro sistema;
- se usan para entrenar un modelo.

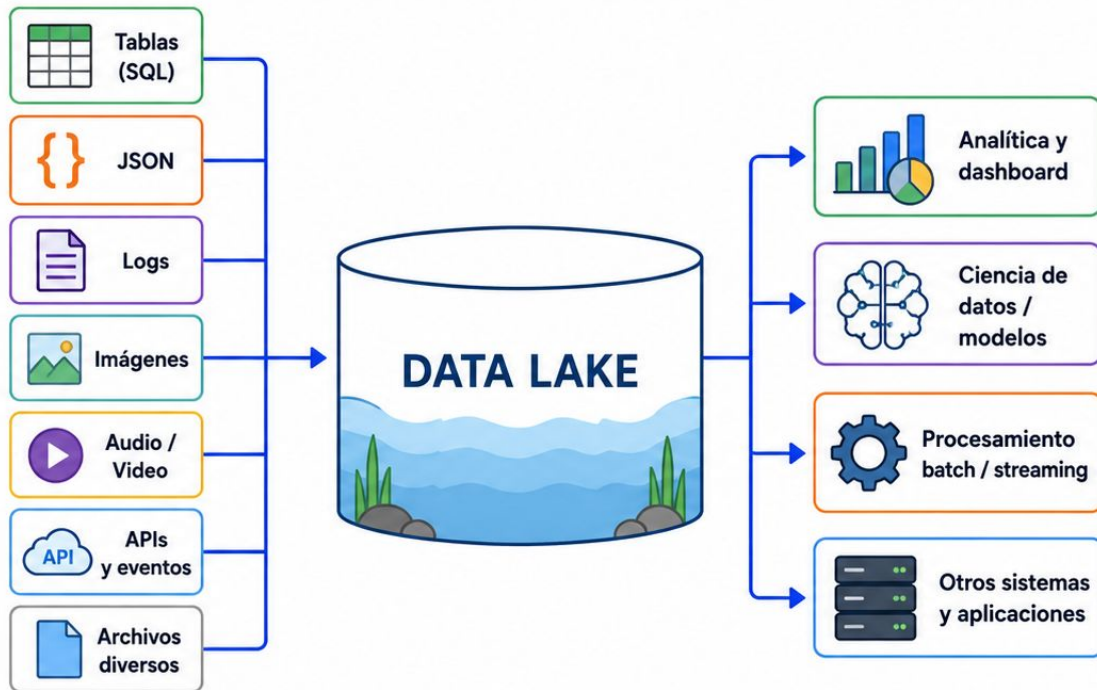


En el Data Lake, el dato no necesita ajustarse a un único esquema antes de ser almacenado.

Ventajas

Flexibilidad y escala

- Admite muchos formatos.
- Conserva datos originales.
- Escala a grandes volúmenes.
- Reduce el costo de almacenamiento.
- Facilita ciencia de datos.
- Permite reprocesar.
- Admite nuevas preguntas.
- Sirve como histórico.



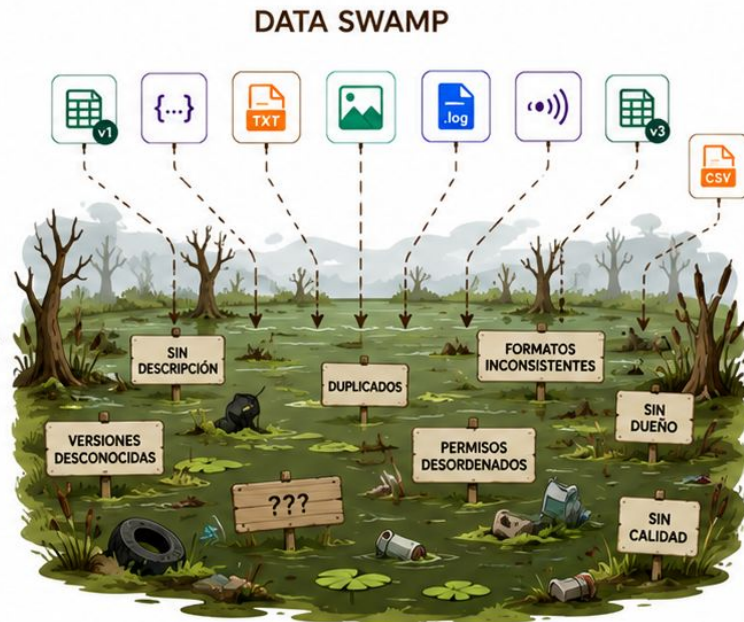
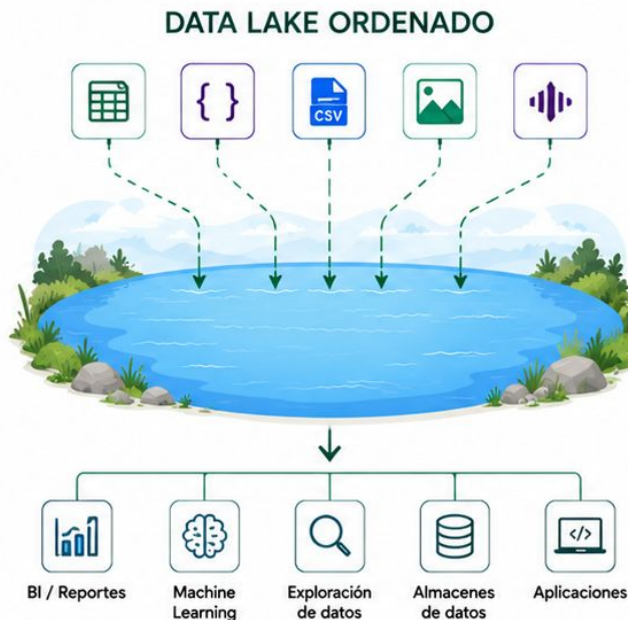
Guardar el dato original permite volver a interpretarlo en el futuro.

Data swamp

Cuando el Data Lake se convierte en un pantano

Un Data Lake puede degradarse si acumula:

- archivos sin descripción;
- datos duplicados;
- formatos inconsistentes;
- versiones desconocidas;
- datos sin dueño;
- permisos desordenados;
- información sin calidad.



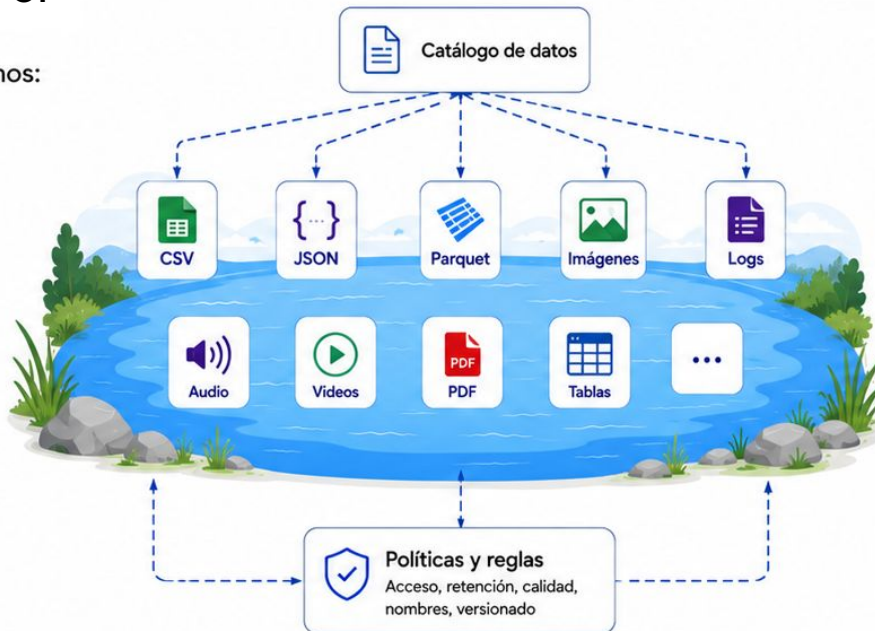
Un Lake sin gobierno puede convertirse en un depósito inutilizable.

Gobernanza

Flexibilidad con control

Para evitar el desorden necesitamos:

- catálogo
- metadatos
- linaje
- propietarios
- permisos
- clasificación
- políticas de retención
- convenciones de nombres
- control de calidad
- versionado



CATÁLOGO / METADATOS



ventas_2025_05.csv

Descripción	Ventas del mes de mayo 2025
Propietario	Equipo Comercial
Origen	Sistema de ventas
Fecha de ingesta	2025-05-15 10:30
Formato	CSV
Clasificación	Confidencial
Retención	2 años
Calidad	Validado
Versión	v2
Linaje	ERP → ETL → Data Lake

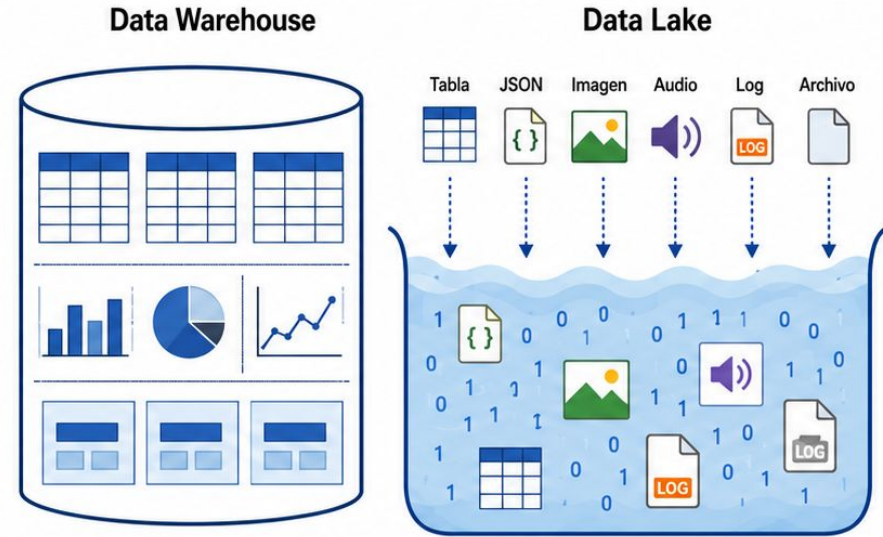


La flexibilidad del Lake necesita reglas de organización.

Comparación

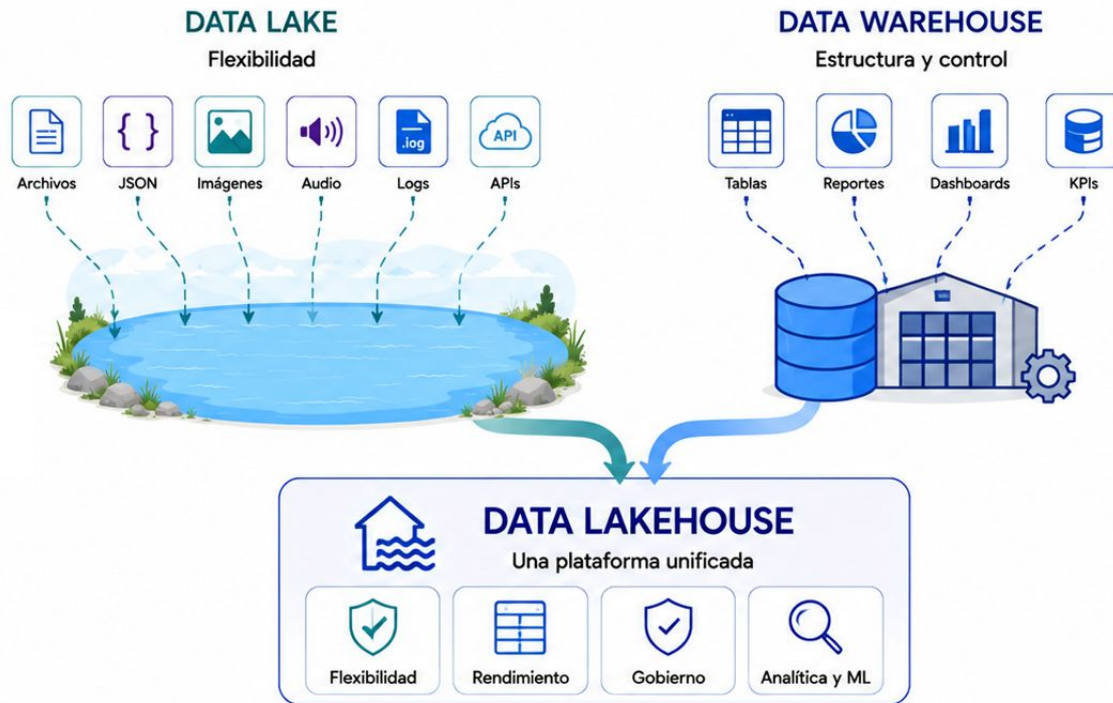
Estructura y flexibilidad

Aspecto	Data Warehouse	Data Lake
Datos	Estructurados	Todos los formatos
Esquema	En escritura	En lectura
Calidad	Alta antes de cargar	Variable
Usuarios	Analistas y BI	Ingeniería y ciencia de datos
Consultas	Optimizadas	Más flexibles
Gobierno	Generalmente alto	Debe diseñarse
Uso	Reportes	Exploración y ML



Warehouse y Lake responden a necesidades diferentes.

Data Lakehouse



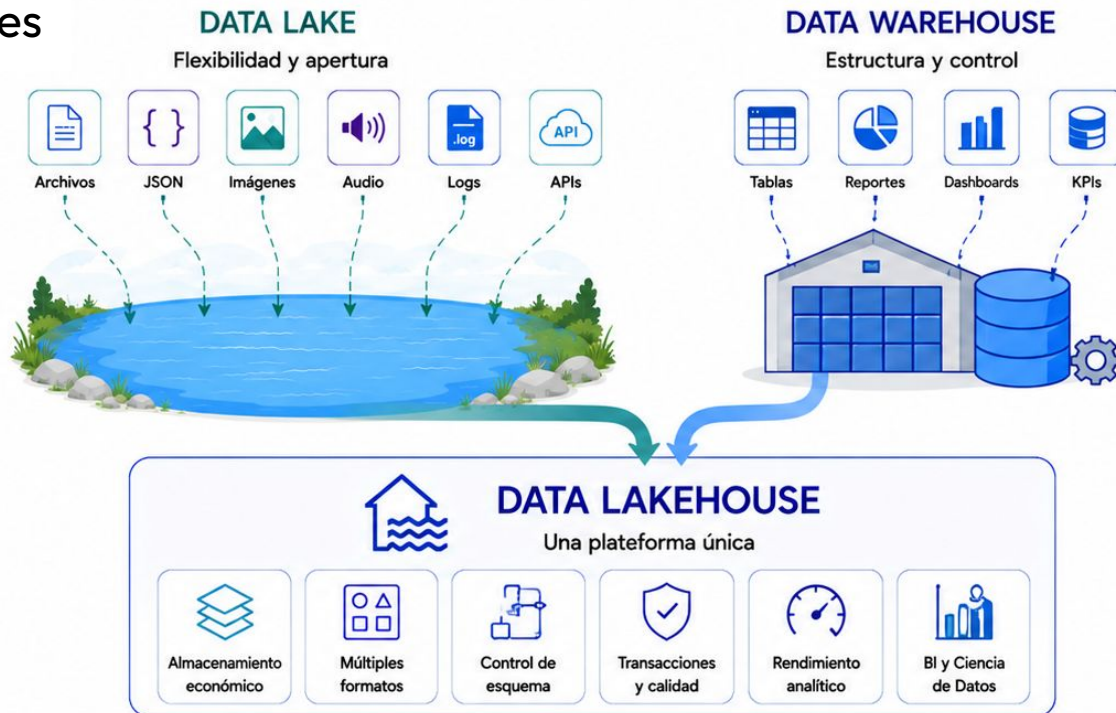
Warehouse y Lake responden a necesidades diferentes.

¿Qué es?

Convergencia de dos enfoques

Un Data Lakehouse busca combinar:

- flexibilidad del Data Lake;
- almacenamiento económico;
- múltiples formatos;
- control de esquema;
- transacciones;
- calidad;
- rendimiento analítico;
- soporte para BI y ciencia de datos.



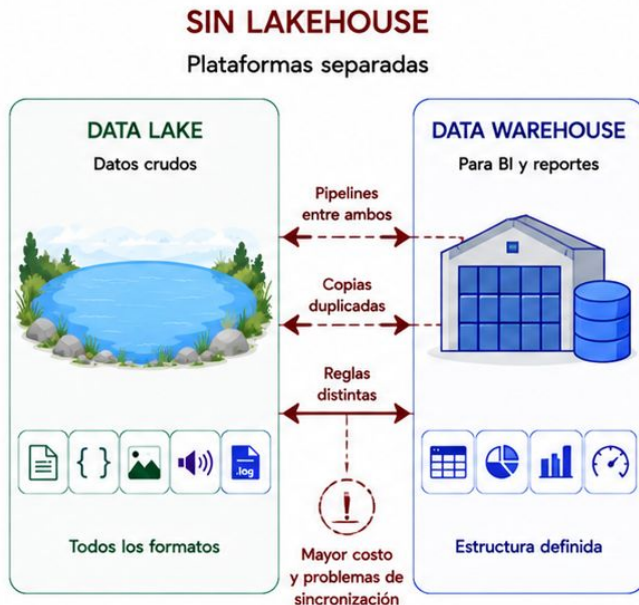
El Lakehouse intenta unir la apertura del Lake con la confiabilidad del Warehouse.

¿Por qué Lakehouse?

Evitar plataformas separadas

Sin Lakehouse, una organización puede terminar con:

- un Lake para datos crudos;
- un Warehouse para BI;
- copias duplicadas;
- pipelines entre ambos;
- reglas distintas;
- mayor costo;
- problemas de sincronización.



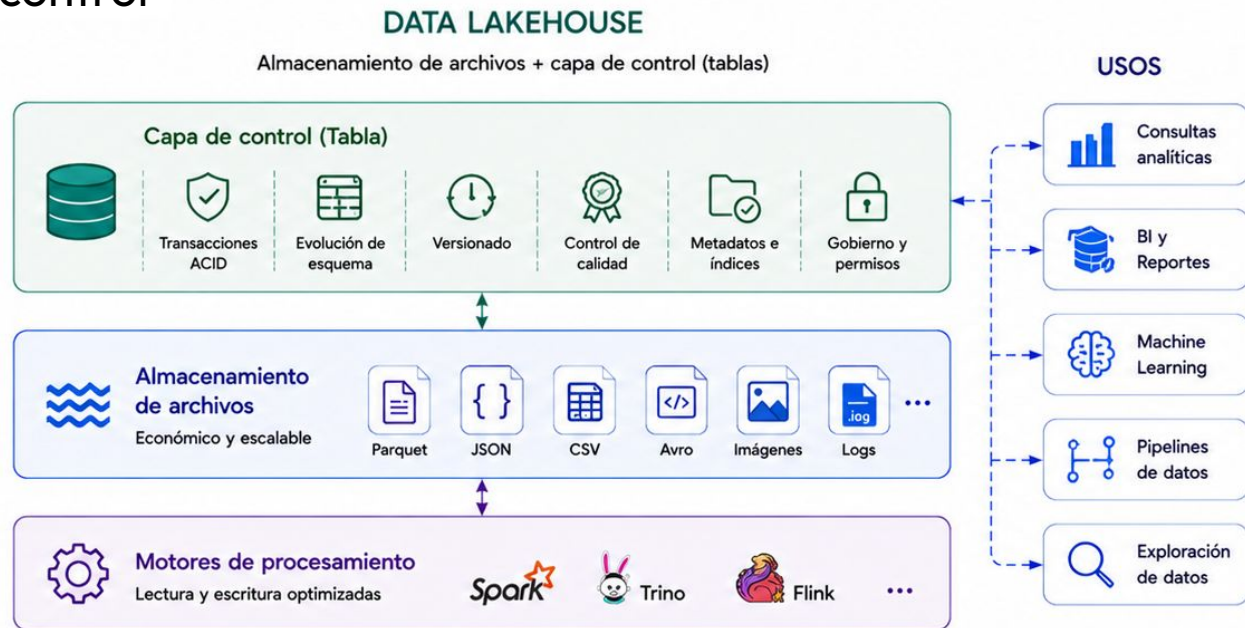
El Lakehouse busca reducir duplicación y fragmentación.

Características

Datos abiertos con mayor control

- Transacciones ACID.
- Evolución de esquema.
- Versionado.
- Control de calidad.
- Consultas analíticas.
- Formatos abiertos.
- Procesamiento batch.
- Integración con ML.
- Soporte para BI.

Tecnologías como **Delta Lake**, **Apache Iceberg** y **Apache Hudi** agregan capacidades de tabla sobre almacenamiento de archivos.



El Lakehouse no es solo una carpeta de archivos: incorpora una capa de control sobre ellos.

Comparación general

Tres enfoques

Aspecto	 WAREHOUSE	 LAKE	 LAKEHOUSE
 Formatos	Estructurados	Todos	Todos
 Gobierno	Alto	Variable	Alto
 Esquema	Escritura	Lectura	Mixto
 BI	Muy bueno	Limitado	Muy bueno
 ML	Posible	Muy bueno	Muy bueno
 Transacciones	Sí	No siempre	Sí
 Flexibilidad	Media	Alta	Alta



El Lakehouse busca ampliar el uso sin separar los datos en plataformas independientes.

Criterio de selección

Evitar complejidad innecesaria

Un Lakehouse puede ser excesivo si:

- el volumen es pequeño;
- los datos son completamente estructurados;
- solo necesitamos reportes simples;
- el equipo es reducido;
- una base relacional resuelve el problema;
- no hay múltiples consumidores;
- no existen necesidades de ML a escala.

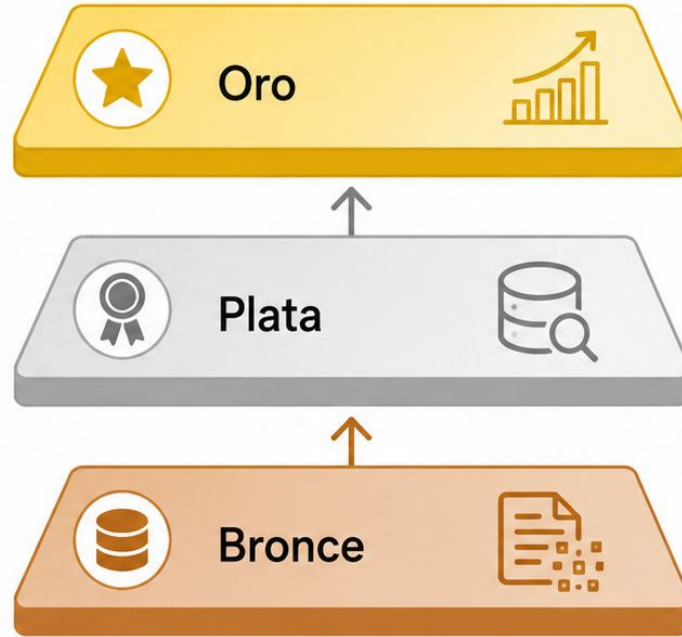


VS.



Una arquitectura moderna no siempre es la arquitectura adecuada.

Arquitectura Medallion



La arquitectura Medallion organiza los datos en capas de calidad creciente.

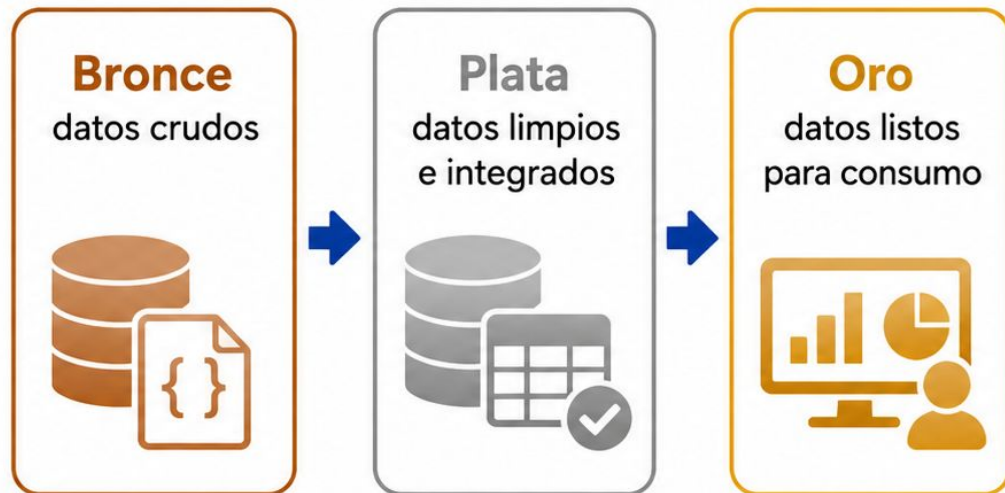
Organización por capas

Separar etapas del dato

La arquitectura Medallion organiza los datos en capas:

- **Bronce:** datos crudos;
- **Plata:** datos limpios e integrados;
- **Oro:** datos listos para consumo.

Cada capa tiene una responsabilidad distinta.



Las capas permiten separar captura, preparación y uso.

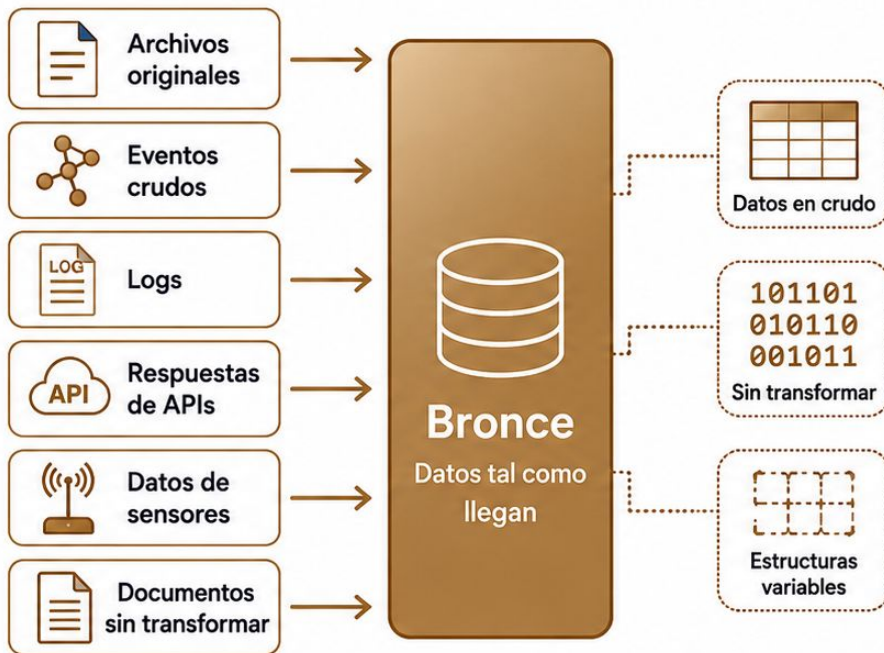
Capa Bronze

Datos tal como llegan

La capa Bronze almacena:

- archivos originales;
- eventos crudos;
- logs;
- respuestas de APIs;
- datos de sensores;
- documentos sin transformar;
- metadatos iniciales.

Puede contener errores, duplicados y estructuras variables.



Bronze conserva la evidencia original.

Capa Plata

Datos limpios, validados e integrados

En la capa Plata se realizan tareas como:

- convertir tipos;
- validar campos;
- eliminar duplicados;
- tratar nulos;
- estandarizar nombres;
- integrar fuentes;
- corregir formatos;
- aplicar reglas de calidad.

Plata convierte datos crudos en datos confiables.



Plata convierte datos crudos en datos confiables.

Capa Oro

Datos preparados para consumo

La capa Oro contiene:

- indicadores;
- agregaciones;
- métricas;
- vistas analíticas;
- tablas para dashboards;
- datos para modelos;
- resultados de inferencia;
- productos de datos.



Oro organiza los datos según una necesidad concreta de uso.

Caso guía por capas

Bronce, Plata y Oro

El mismo caso de experimentos de IA representado en cada capa.

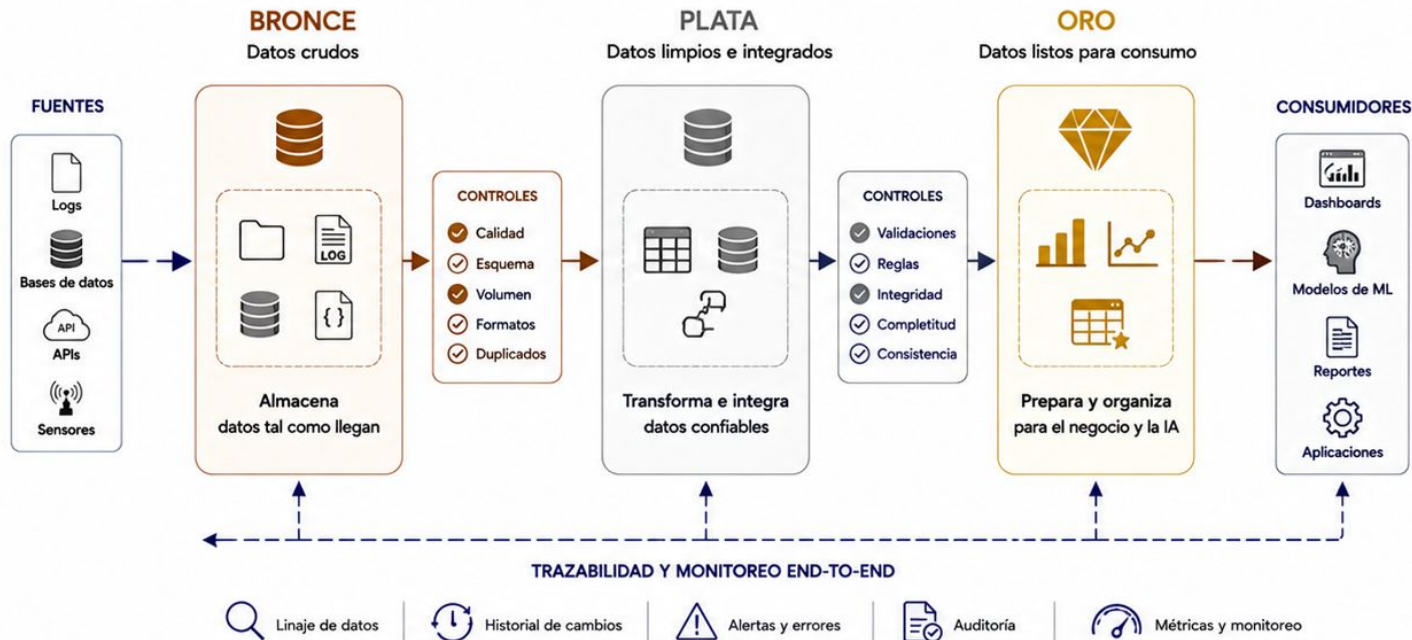


El mismo dato cambia de forma según la etapa en la que se encuentra.

Ventajas del enfoque por capas

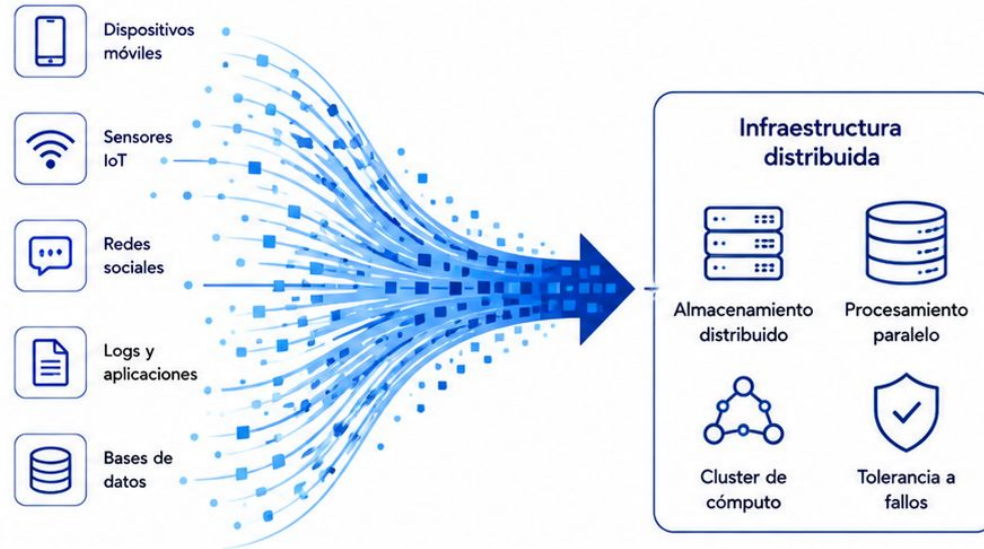
Trazabilidad y mantenimiento

- Separación de responsabilidades.
- Reprocesamiento.
- Auditoría.
- Control de calidad.
- Evolución independiente.
- Detección de errores.
- Reutilización.
- Escalabilidad.



Las capas hacen visible qué se hizo con cada dato.

Introducción a Big Data



Big Data aparece cuando el volumen, la velocidad o la variedad superan la capacidad de las herramientas tradicionales.

¿Qué es Big Data?

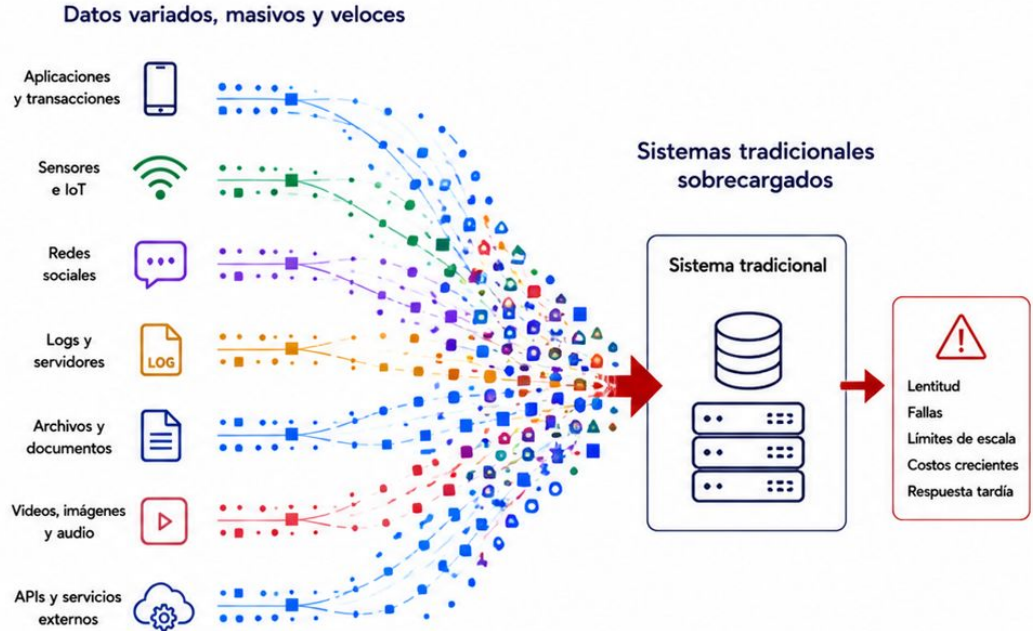
Más que muchos datos

Big Data aparece cuando el volumen, la velocidad o la complejidad de los datos superan lo que las herramientas tradicionales pueden manejar de forma razonable.

No se trata solamente de cantidad.

También importa:

- cómo llegan;
- en qué formatos;
- con qué calidad;
- cómo se procesan;
- qué valor podemos obtener.

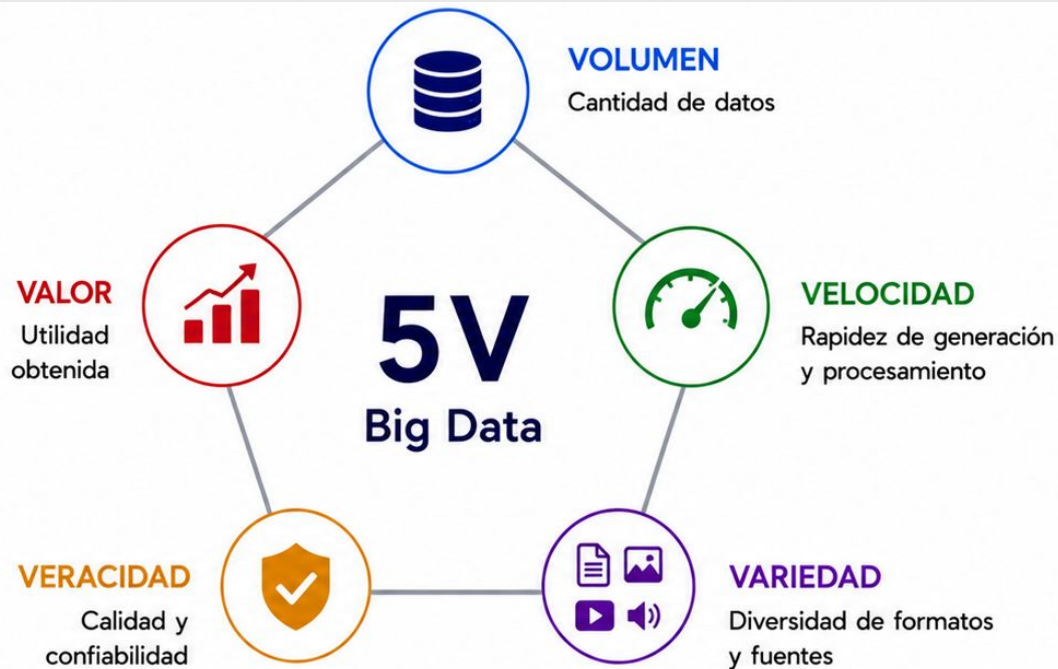


Big Data es un problema de escala, diversidad y procesamiento.

Las cinco V

Características del Big Data

- **Volumen:** cantidad de datos.
- **Velocidad:** rapidez de generación y procesamiento.
- **Variedad:** diversidad de formatos y fuentes.
- **Veracidad:** calidad y confiabilidad.
- **Valor:** utilidad obtenida.



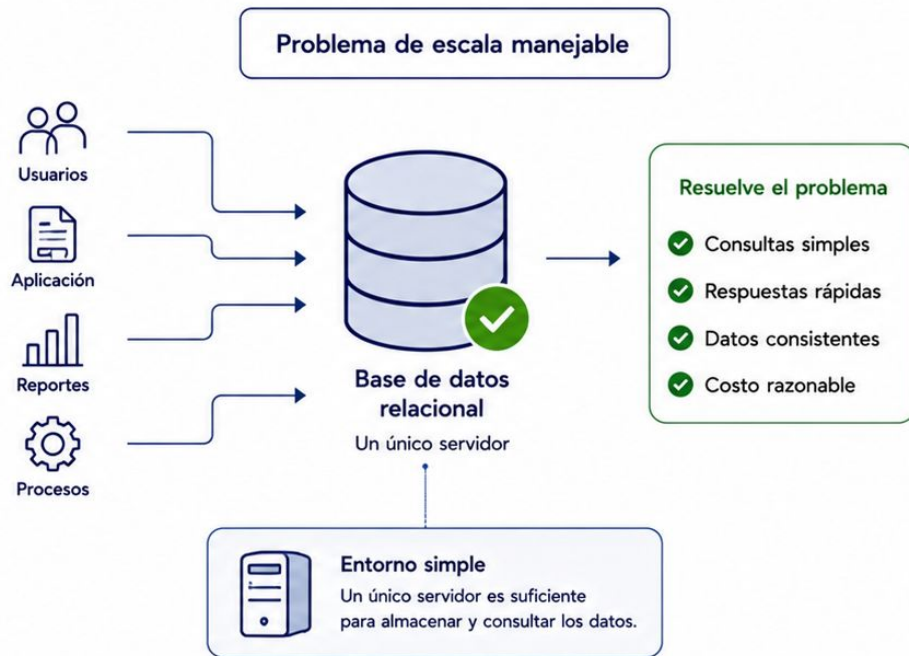
Tener muchos datos no sirve si no son confiables ni generan valor.

Criterio de escala

Evitar sobredimensionar

Un problema puede no requerir Big Data si:

- cabe en una base relacional;
- el volumen es manejable;
- las consultas son simples;
- la carga es estable;
- no hay múltiples fuentes;
- no se necesita distribución;
- un único servidor resuelve el caso.



Big Data debe ser una necesidad, no una etiqueta.

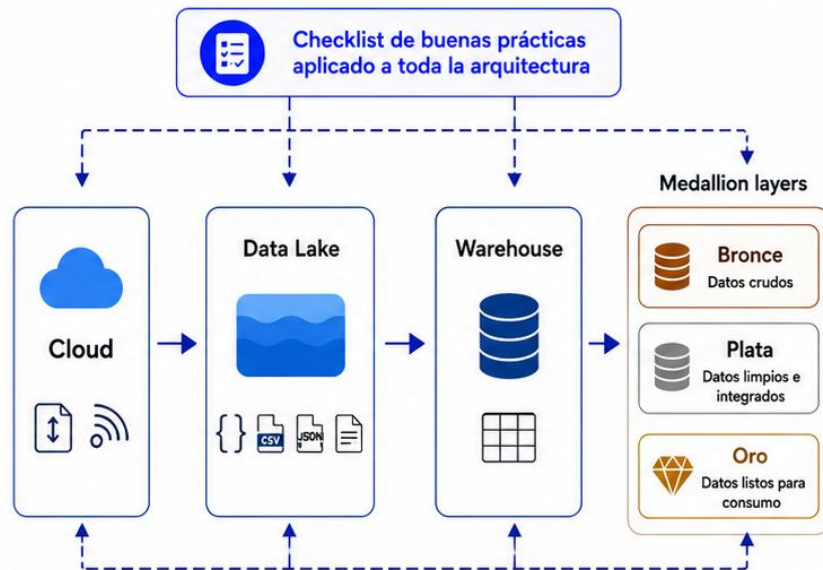


Para ir cerrando..

Buenas prácticas

Criterios para diseñar arquitecturas de datos

- ✓ Empezar por los requerimientos y no por las herramientas.
- ✓ Separar claramente las cargas operativas de las analíticas.
- ✓ Elegir la arquitectura más simple que resuelva el problema.
- ✓ Conservar los datos originales cuando sea necesario reprocesar.
- ✓ Documentar las transformaciones entre capas.
- ✓ Incorporar metadatos, calidad y trazabilidad desde el inicio.
- ✓ Controlar costos de almacenamiento, procesamiento y transferencia.
- ✓ Evitar adoptar cloud, Lakehouse o Big Data solo por tendencia.
- ✓ Diseñar pensando en evolución, seguridad y mantenimiento.
- ✓ Justificar cada componente y su responsabilidad.

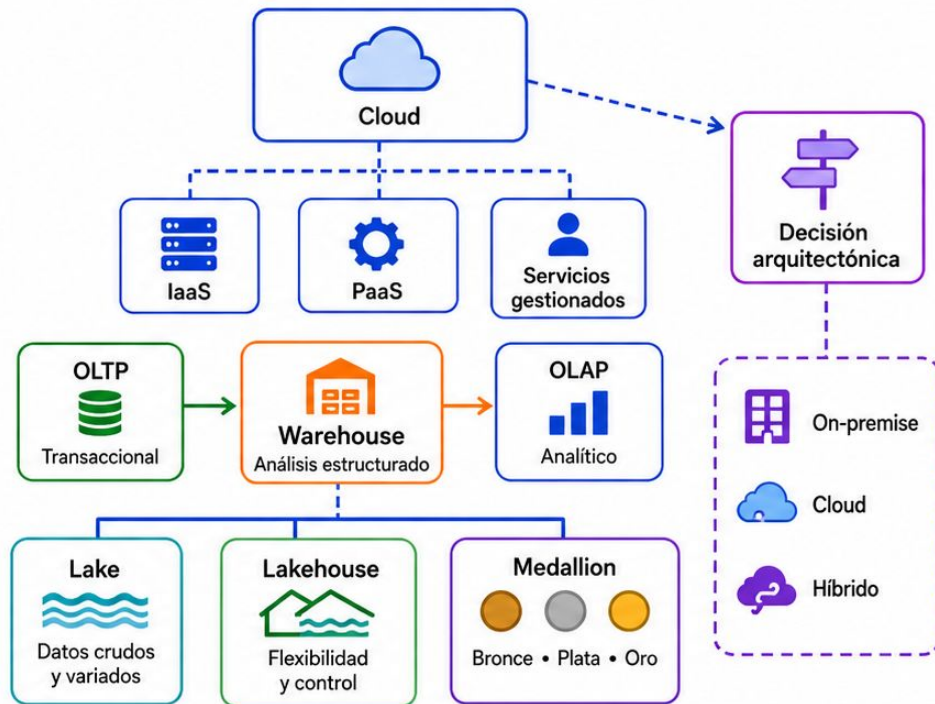


Una buena arquitectura no es la más compleja, sino la que puede comprenderse, operarse y evolucionar.

Resumen de la clase

Ideas principales

- Una base de datos forma parte de una arquitectura mayor.
- Cloud permite consumir infraestructura bajo demanda.
- IaaS, PaaS y servicios gestionados distribuyen responsabilidades.
- On-premise, cloud e híbrido responden a necesidades distintas.
- OLTP y OLAP representan cargas diferentes.
- Data Warehouse organiza análisis estructurado.
- Data Lake conserva datos variados y crudos.
- Data Lakehouse combina flexibilidad y control.
- Medallion separa Bronce, Plata y Oro.
- Big Data implica volumen, velocidad, variedad, veracidad y valor.
- La arquitectura debe elegirse según el problema.



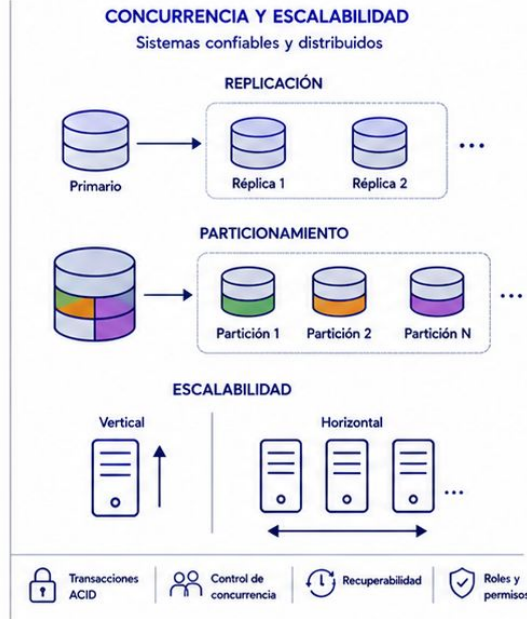
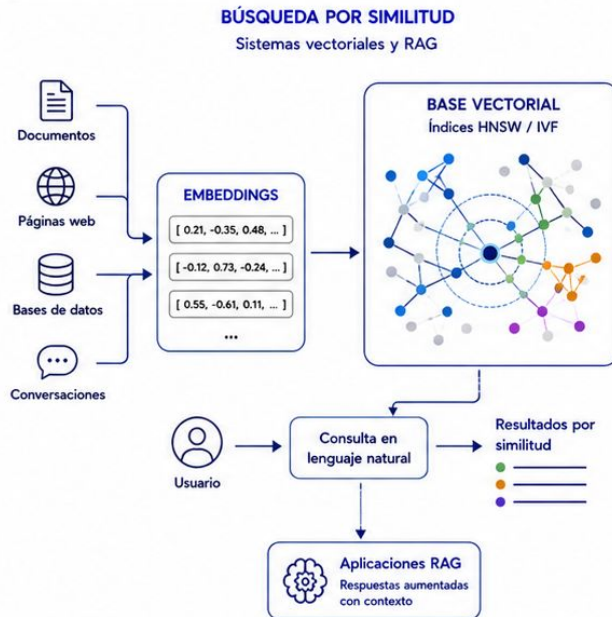
La mejor arquitectura es la que organiza los datos con claridad, control y propósito.

Próxima clase

Bases Vectoriales, Concurrency y Escalabilidad

En la próxima clase vamos a trabajar sobre:

- embeddings;
- búsqueda por similitud;
- bases de datos vectoriales;
- índices HNSW e IVF;
- aplicaciones en sistemas RAG;
- transacciones y propiedades ACID;
- control de concurrencia;
- recuperabilidad;
- replicación;
- particionamiento;
- escalabilidad vertical y horizontal;
- roles y permisos.



Hoy organizamos la infraestructura de datos.

La próxima clase veremos cómo buscar por significado y cómo sostener sistemas concurrentes y escalables.



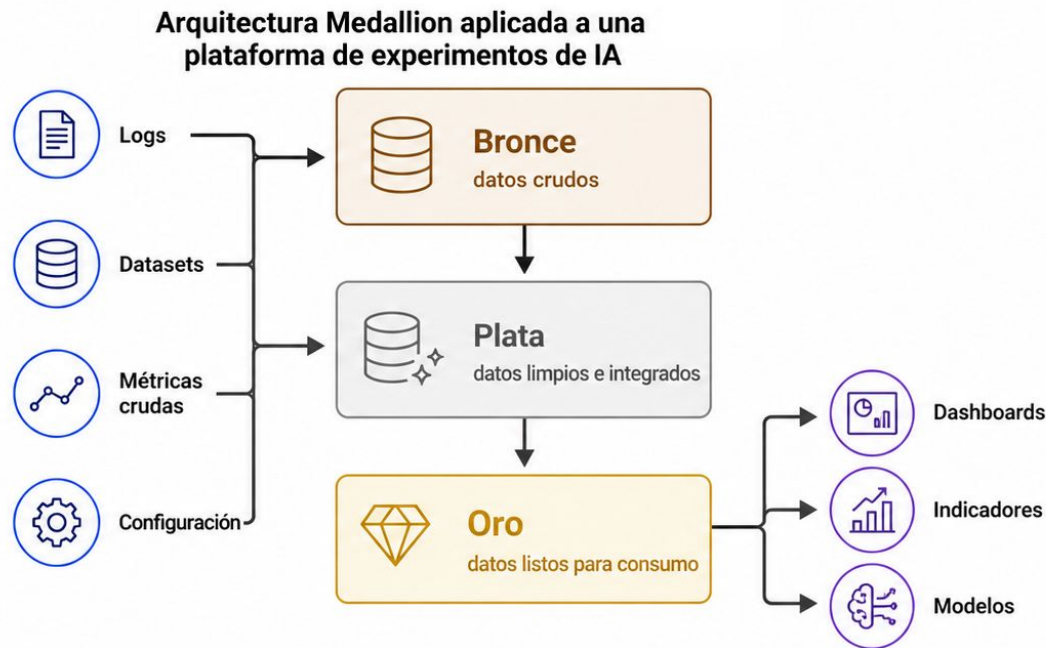
Enlace a la encuesta



Muchas gracias ¿Dudas?

Práctica

Diseñar una arquitectura por capas



En la práctica vamos a decidir qué guardar en cada capa y cómo transformar los datos hasta llegar al consumo.

Objetivo y escenario

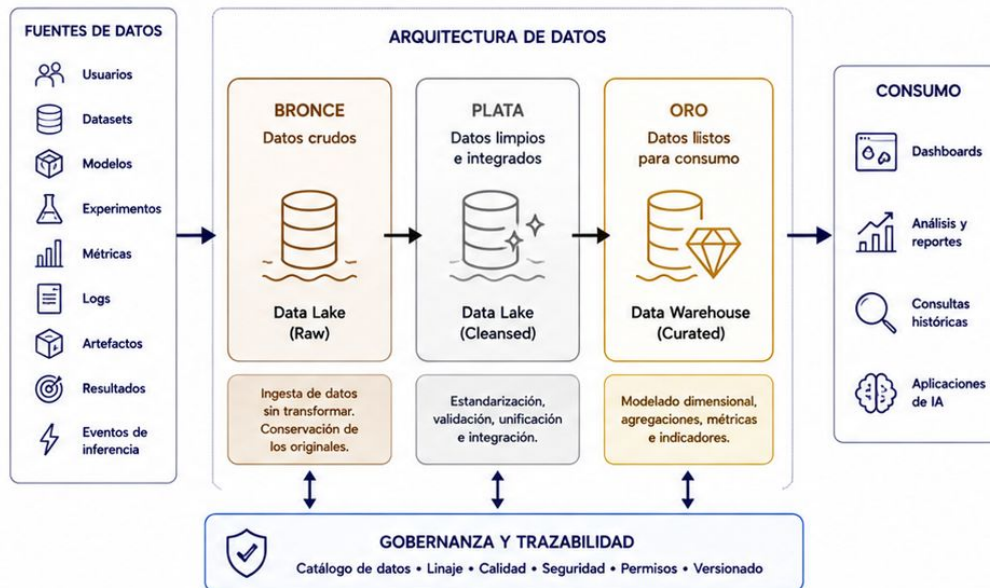
Diseñar una plataforma de datos para IA

Vamos a tomar el caso guía y diseñar una arquitectura capaz de manejar:

- usuarios
- datasets
- modelos
- experimentos
- métricas
- logs
- artefactos
- resultados
- eventos de inferencia

La organización necesita:

- ✓ registrar experimentos
- ✓ consultar históricos
- ✓ conservar datos originales
- ✓ escalar el almacenamiento
- ✓ comparar métricas
- ✓ mantener trazabilidad
- ✓ generar dashboards



Cada requerimiento debe reflejarse en una decisión de arquitectura.

Fuentes, base operativa y Bronce

Identificar el origen y conservar el dato

Fuentes

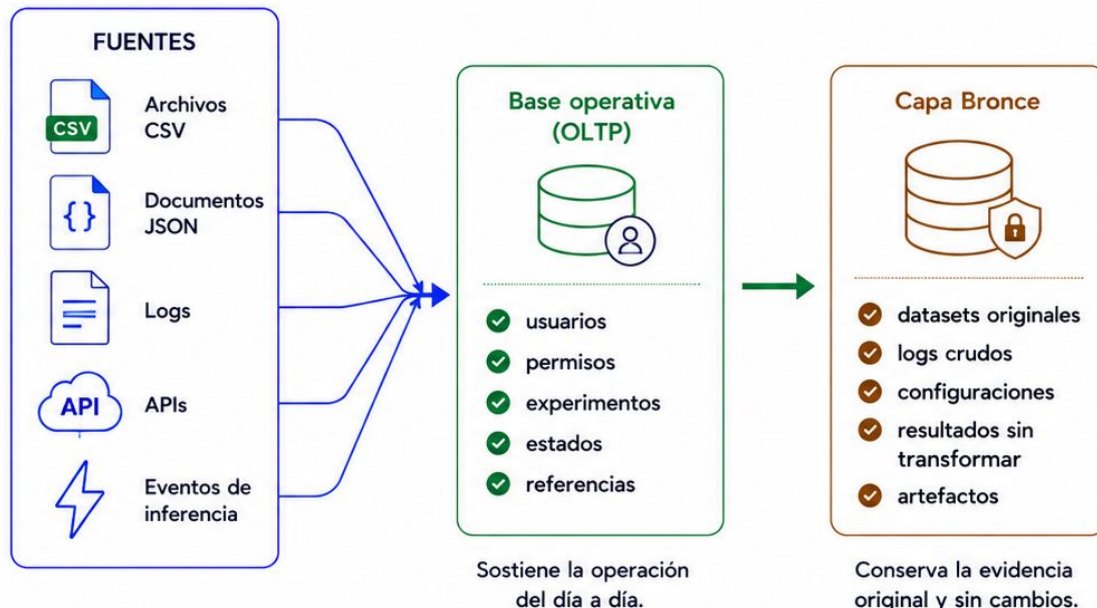
- archivos CSV;
- documentos JSON;
- logs;
- APIs;
- eventos de inferencia.

Base operativa — OLTP

- usuarios;
- permisos;
- experimentos;
- estados;
- referencias.

Capa Bronce

- datasets originales;
- logs crudos;
- configuraciones;
- resultados sin transformar;
- artefactos.



Antes de transformar, debemos saber qué llega, qué sostiene la operación y qué debe conservarse.

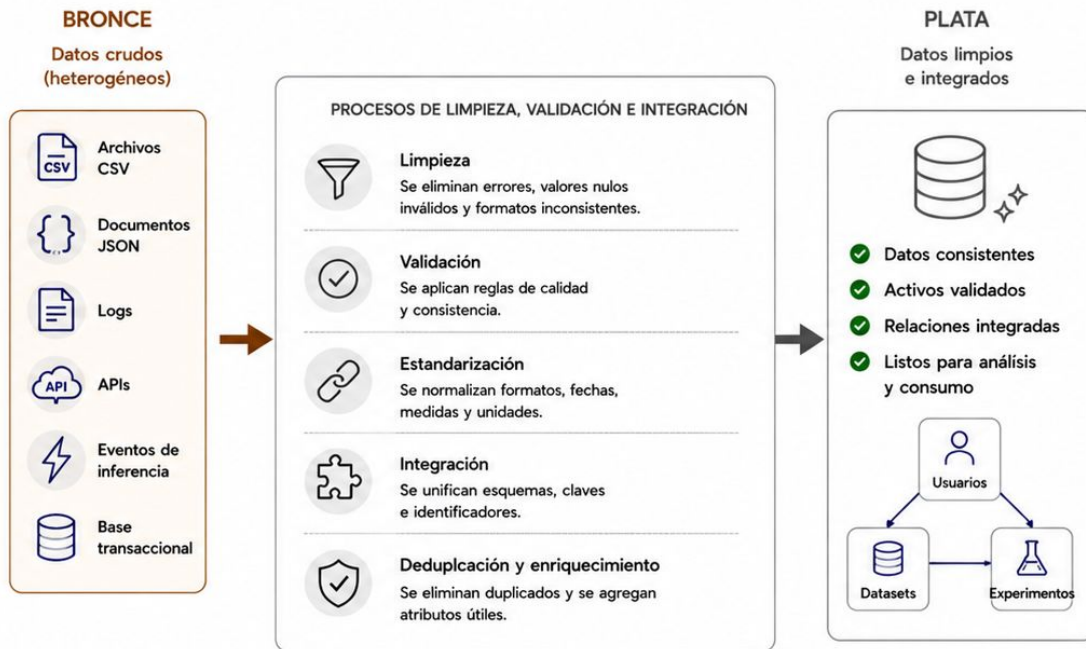
Capa Plata

Limpiar, validar e integrar

La capa Plata podría contener:

- métricas estandarizadas;
- fechas normalizadas;
- identificadores unificados;
- experimentos validados;
- registros sin duplicados;
- modelos versionados;
- relaciones consistentes entre usuarios, datasets y experimentos.

En esta etapa se corrigen diferencias de formato, se aplican reglas de calidad y se integran las distintas fuentes.



Plata convierte datos crudos en datos consistentes y confiables.

Capa Oro

Preparar datos para análisis y consumo

La capa Oro podría ofrecer:

- accuracy promedio por modelo;
- mejores experimentos;
- costo por entrenamiento;
- uso de recursos por usuario;
- evolución temporal de métricas;
- datasets más utilizados;
- modelos candidatos a despliegue;
- indicadores para dashboards.

Los datos se organizan según las preguntas que deben responder los analistas, aplicaciones o modelos.



Oro transforma datos confiables en productos listos para utilizar.

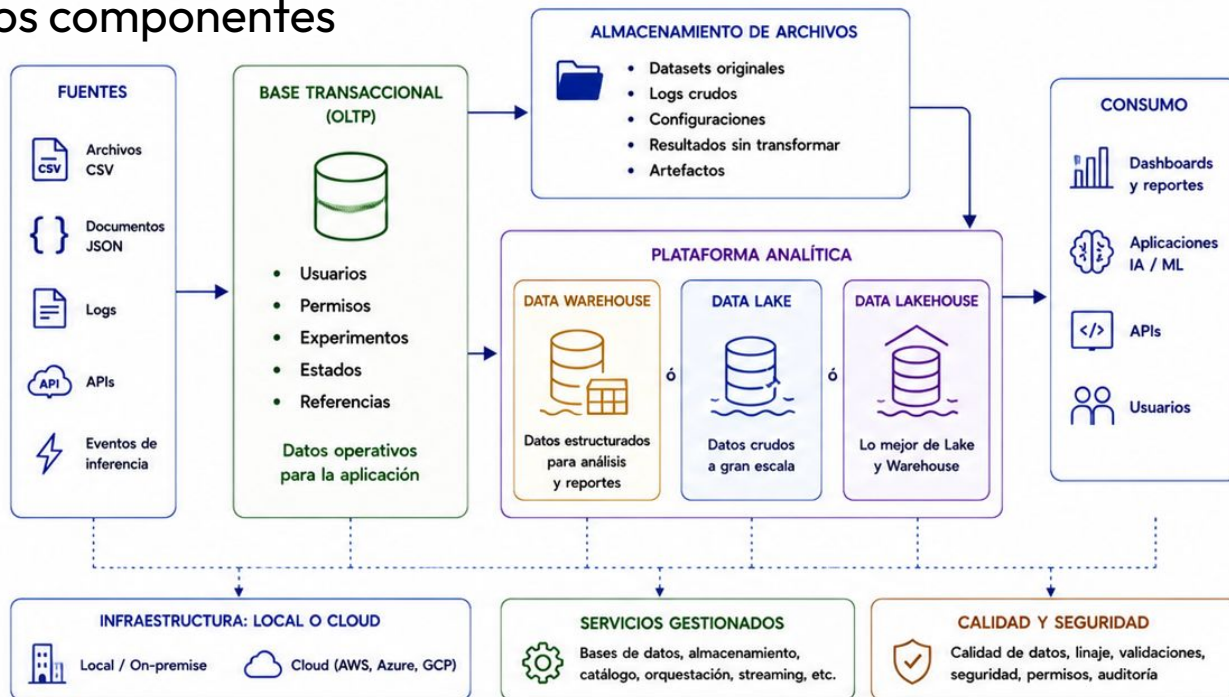
Integrar la arquitectura

Definir cómo se conectan los componentes

Decidir:

- qué datos permanecen en la base transaccional;
- qué datos se almacenan como archivos;
- si necesitamos un Data Warehouse, Data Lake o Data Lakehouse;
- qué componentes serán locales o cloud;
- qué servicios conviene gestionar;
- qué controles de calidad y seguridad aplicar.

Finalmente, vamos a representar el flujo completo y la función de cada componente.



Una buena arquitectura no solo se dibuja: también se justifica.